

**THE UNIVERSITY OF DANANG
UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF ADVANCED SCIENCE AND TECHNOLOGY**



CAPSTONE PROJECT

RESEARCH AND IMPLEMENTATION OF AN AUTOMATED BOLT SORTING SYSTEM USING FLEX FEEDER AND ROBOTIC MANIPULATOR

**NGUYEN ANH TUAN
TRINH CONG THANH
HOANG THI DUYEN**

**Supervisor: DR. NGUYEN QUANG NHU QUYNH
Co-supervisors: ENG. NGUYEN KHANH TRINH**

Danang, June 2026

**THE UNIVERSITY OF DANANG
UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF ADVANCED SCIENCE AND TECHNOLOGY**



CAPSTONE PROJECT

RESEARCH AND IMPLEMENTATION OF AN AUTOMATED BOLT SORTING SYSTEM USING FLEX FEEDER AND ROBOTIC MANIPULATOR

**NGUYEN ANH TUAN
TRINH CONG THANH
HOANG THI DUYEN**

**Supervisor: DR. NGUYEN QUANG NHU QUYNH
Co-supervisors: ENG. NGUYEN KHANH TRINH**

Danang, June 2026

CAPSTONE PROJECT REQUIREMENTS

No.	Student name	Student ID	Class	Major
1	Trinh Cong Thanh	123210175	21PFIEV2	Industrial informatics
2	Nguyen Anh Tuan	123210180	21PFIEV2	Industrial informatics
3	Hoang Thi Duyen	123210159	21PFIEV2	Industrial informatics

1. *Capstone project title:*

Research and Implementation of an Automated Bolt Sorting System using Flex Feeder and Robotic Manipulator.

2. *Project topic:*

This project investigates the design and implementation of an automated bolt sorting system using Flex Feeder technology and an industrial robot. The system detects and picks only the target M8×15 mm bolts from a cluttered feeder tray containing many similar fasteners and noise objects. The vision system identifies the bolt position and orientation, then sends coordinates to the robot for picking and placement.

3. *Content of the explanations and calculations:*

Division of tasks:

No.	Student name	Content
1	Trinh Cong Thanh	Develop the machine vision module for detecting M8×15 mm target bolts in mixed batches. Collect and label datasets for model training. Train and evaluate the YOLOv8 Segmentation model. Set up the industrial camera and perform camera calibration for coordinate conversion and robot guidance. Implement image processing algorithms for localization and orientation estimation.
2	Nguyen Anh Tuan	Study the operating principles of the Flex Feeder and PLC-based control system. Develop the PLC control program for feeder operation and sequence control. Implement communication and synchronization between the PLC and robot controller. Integrate feeder control functions, including spreading, flipping, and purge operations.
3	Hoang Thi Duyen	Develop robot programs for vision-guided pick-and-place operations. Configure communication between the robot controller and Vision PC. Implement coordinate transformation and robotic grasping functions. Integrate the industrial robot with the PLC and vision system. Optimize robot motion and cycle execution.
4	Shared Responsibilities	Study the overall architecture and operating workflow of the system. Participate in system integration, testing, troubleshooting, and performance optimization. Contribute to the preparation of the technical report and project documentation.

4. *Supervisor:*

No.	Supervisor	Content
1	Dr. Nguyen Quang Nhu Quynh	- Provide continuous supervision and guidance throughout the project implementation process. - Provide relevant reference materials and technical documentation.
2	Eng. Nguyen Khanh Trinh	

5. *Date of assignment:* 09/03/2026

6. *Date of completion:* 31/05/2026

Da Nang , June 25th 2026
Supervisor

Dr. Nguyen Quang Nhu Quynh

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM

Độc lập – Tự do – Hạnh phúc

GIẤY THỎA THUẬN

V/v Sử dụng, khai thác nội dung tài liệu nộp lưu chiểu

Tôi tên là: Nguyễn Anh Tuấn, Trịnh Công Thành, Hoàng Thị Duyên

Là tác giả tài liệu: Research and Implementation of an Automated Bolt Sorting System using Flex Feeder and Robotic Manipulator.

Tôi làm bản thỏa thuận này với Trung tâm Học liệu và Truyền thông, Trường Đại học Bách khoa, Đại học Đà Nẵng như sau:

Tôi đồng ý cho phép Trung tâm Trung tâm Học liệu và Truyền thông, Trường Đại học Bách khoa, Đại học Đà Nẵng toàn quyền sử dụng và khai thác nội dung tài liệu vào mục đích phục vụ nghiên cứu khoa học, giảng dạy và học tập.

Tôi cam kết sẽ không có bất kỳ khiếu nại nào liên quan đến tài liệu trên. Nếu sai tôi hoàn toàn chịu trách nhiệm trước pháp luật.

Đà Nẵng, ngày 25 tháng 6 năm 2026

Tác giả

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

[illegible]

ABSTRACT

Capstone Project: Research and Implementation of an Automated Bolt Sorting System using Flex Feeder and Robotic Manipulator

Modern industrial assembly lines increasingly require flexible automation solutions capable of handling small mechanical components that are presented in random orientations and mixed batches. In many practical production environments, bolts, and similar fasteners are supplied in cluttered conditions, which makes direct robotic picking difficult when conventional feeding systems are used. To address this challenge, this project proposes an automated bolt sorting system that combines Flex Feeder technology, industrial machine vision, a Nachi MZ04 industrial robot, and a Keyence KV-5500 PLC into a unified robotic handling cell.

The proposed system is designed to detect and pick only the target M8×15 mm bolts from a mixed feeder tray containing multiple fasteners and other noise objects. The Flex Feeder is used to redistribute and orient the parts on the tray, while the industrial camera captures images of the working area for vision-based detection. The machine vision software identifies the target bolt, determines its position and orientation, and transmits the corresponding coordinates to the robot controller. The robot then executes the picking motion and places the target part into a designated output location. When no valid target bolt is detected, the feeder is activated again to create a new arrangement of parts for the next inspection cycle.

This project focuses on the integration of mechanical feeding, vision processing, robotic motion, and industrial communication rather than on isolated subsystems. By combining these components into a coordinated automation platform, the system aims to improve flexibility in part handling, reduce dependence on manual sorting, and support reliable operation in real industrial environments. The expected outcome is a practical and extensible solution for automated handling of small fasteners in cluttered conditions, with potential application in smart manufacturing and flexible production lines.

ACKNOWLEDGEMENT

During the preparation of this report, our group received timely guidance and many practical suggestions that helped us clarify the problem statement, refine the technical direction, and structure the initial plan for the project. We would like to express our sincere appreciation to Dr. Nguyen Quang Nhu Quynh for her close supervision, clear feedback, and support in shaping our research objectives and methodology for the first phase.

We would also like to thank Eng. Nguyen Khanh Trinh for sharing hands-on engineering experience and advising us on system feasibility, integration logic, and key technical considerations for a Flex Feeder–vision–robot solution. In addition, we are grateful to Mr. Phan Van Linh (Director of ITSC) for his encouragement and for helping our group better understand practical deployment requirements and project planning.

Finally, we would like to thank IDEA TECHNOLOGY SOLUTIONS JSC for creating favorable conditions and support that enabled us to complete the deliverables on time.

Despite our efforts, this report may still contain limitations. We sincerely welcome further comments and suggestions so that the next phases of the project can be carried out more effectively.

Thank you all sincerely!

Danang, June, 25th , 2026

DECLARATION

We confirm that this report for the capstone project titled “Research and Implementation of an Automated Bolt Sorting System Using Flex Feeder and Robotic Manipulator” has been prepared by our group based on our own study and work completed in the first phase of the project. This report was developed under the academic supervision of Dr. Nguyen Quang Nhu Quynh, with technical guidance from Eng. Nguyen Khanh Trinh and Mr. Phan Van Linh (Director of ITSC). All external references used in this report are properly cited in the reference list. The information, analysis, and preliminary results presented here are truthful to the best of our knowledge and do not infringe upon any copyrights or intellectual property rights. We take full responsibility for the content of this report.

Danang, June, 25th , 2026

Implementing students

Trinh Cong Thanh

Nguyen Anh Tuan

Hoang Thi Duyen

TABLE OF CONTENTS

ABSTRACT	i
ACKNOWLEDGEMENT	ii
DECLARATION	iii
TABLE OF CONTENTS	iv
LIST OF FIGURES	vi
LIST OF TABLE	vii
ABBREVIATIONS	viii
CHAPTER 1: CHALLENGES AND SOLUTIONS FOR AUTOMATED BOLT SORTING	1
1.1 Project overview.....	1
1.1.1 Problem statement.....	1
1.1.2 Overview of existing solutions	1
1.1.3 Compare the solutions	4
1.2 Proposed system.....	4
1.2.1 Proposed solution.....	4
1.2.2 Geometric and surface characteristics of the research object.....	5
1.2.2.1 Geometric characteristics	5
1.2.2.2 Surface characteristics	6
1.2.3 Hierarchical architecture and overall block diagram.....	6
1.3 Chapter summary	7
CHAPTER 2: THEORETICAL FOUNDATION AND RELATED COMPONENTS	8
2.1 System requirements specification.....	8
2.1.1 Spatial positioning accuracy	8
2.1.2 Rotational orientation accuracy	8
2.1.3 Reliability & pick success rate	9
2.2 Theoretical foundation	9
2.2.1 Theoretical basis of image processing and deep learning	10
2.2.2 YOLOv8 Segmentation algorithm YOLOv8	11
2.2.3 PCA Algorithm for determining rotation angle.....	13
2.2.4 Theoretical basis of coordinate calibration.....	14
2.2.5 Communication interface theory	14

2.3 Related equipment.....	15
2.3.1 Nachi MZ04 robot	15
2.3.2 PLC control equipment.....	18
2.3.3 HMI VT5-W07	19
2.3.4 Flex feeder	19
2.3.5 Industrial lighting and camera system	20
2.3.6 Power supply units and safety equipment	21
2.3.7 System assembly model.....	22
2.3.8 KV-Studio software	22
2.3.9 HMI VT-Studio software.....	22
2.3.10 FD on desk software	23
2.3.11 WinForms (C#) interface development platform	24
2.4 Chapter summary	24
CHAPTER 3. SYSTEM DESIGN AND CONTROL ARCHITECTURE	25
3.1 Overall system design	25
3.2 Machine vision system design	27
3.3 Coordinate transformation design	31
3.4 Flex feeder operational strategy design	33
3.4.1 Component spreading mode	33
3.4.2 Purge mode	34
3.5 Interface and data transmission design	35
3.6 Supervision and control interface design (WinForm application).....	38
3.6.1 Software architecture	39
3.6.2 Main interface layout	40
3.6.3 Control and monitoring functions.....	41
3.6.4 Robot coordinate data management.....	41
3.6.5 Logging and alarm functions	42
3.7 Robot arm control programming.....	43
3.8 Chapter summary	45
CHAPTER 4. SYSTEM DEVELOPMENT AND EXPERIMENTAL EVALUATION	46
4.1 Experimental setup and system integration	46
4.2 Experimental results and technical parameter	46

4.2.1 Positioning accuracy and sorting success rate evaluation	46
4.2.2 Flex feeder component separation efficiency evaluation	48
4.2.3 Stability evaluation	48
4.3 Project Conclusion and future directions	49
4.3.1. Attained results and system limitations	49
4.3.2. Future research and development scope in the future	49
APPENDIX	50
REFERENCES	53

LIST OF FIGURES

Figure 1.1 Vibratory bowl feeder for automatic part feeding.	2
Figure 1.2 Conveyor-Based part sorting system.	2
Figure 1.3 Industrial machine vision system for automated inspection.	3
Figure 1.4 Layered functional architecture of the system.	6
Figure 1.5 Overall block diagram.....	7
Figure 2. 1 Illustration of gripper orientation alignment and angular error during picking.....	8
Figure 2. 2 Execution process	10
Figure 2. 3 Operational concept of the YOLO algorithm	12
Figure 2. 4 Result of the MinAreaRect Algorithm.....	13
Figure 2. 5 TCP Socket interface concept between controller and vision system	15
Figure 2. 6 Nachi Robot coordinate systems.....	16
Figure 2. 7 Tool coordinate system	17
Figure 2. 8 Structural faces of the CFD controller	17
Figure 2. 9 Control programming via ASCII file and executable work program format .	18
Figure 2. 10 PLC keyence kv-5500	19
Figure 2. 11 HMI VT5-W07	19
Figure 2. 12 HG-KR23B servo motor	20
Figure 2. 13 MR-J4-20A servo drive	20
Figure 2. 14 Camera IMC3616UC	21
Figure 2. 15 Illumination Light TH2-83X75SW.....	21
Figure 2. 16 MCB Panasonic 1P	22
Figure 2. 17 OMRON S8VK-S03024 power supply	22
Figure 2. 18 MCB Panasonic CP40-BA 2P.....	22
Figure 2. 19 3D CAD Assembly model of the system	22
Figure 2. 20 KV Studio software interface showing ladder logic code	23
Figure 2. 21 VT Studio software interface showing HMI screen development.....	23

Figure 2. 22 FD on Desk software interface panel.....	23
Figure 3.1 Overall system algorithm flowchart	26
Figure 3.2 Data annotation process interface in Roboflow	28
Figure 3.3 Dataset properties and data augmentation configurations	29
Figure 3.4 A graph illustrating model convergence (Training/Validation loss curves)....	29
Figure 3.5 PCA processing result.....	31
Figure 3.6 Calibrated camera field of view and robotic arm taking points on physical tray	32
Figure 3.7 Ladder logic control program for Up Feeder component distribution.....	33
Figure 3.8 Ladder logic control program for Down Feeder component spreading.....	34
Figure 3.9 Ladder logic control program for cleaning Down Feeder components	35
Figure 3.10 Communication settings panel for KV studio IP and port configuration	36
Figure 3.11 PLC communication condition setup page and Ethernet communication	36
Figure 3.12 Ethernet communication setup in VT Studio.....	37
Figure 3.13 Communication settings for connecting with the PLC	37
Figure 3.14 System data transmission diagram.....	38
Figure 3.15 Main window layout of the WinForm application.....	39
Figure 3.16 Module architecture block diagram of the WinForm application.....	40
Figure 3.17 Geometric layout of the main user interface.....	41
Figure 3.18 Control button region and configuration dialog access buttons	41
Figure 3.19 Detection coordinate list table and robot transmission status flags	42
Figure 3.20 Status bar location and system alarm region.....	43
Figure 3.21 Start FD on Desk.....	43
Figure 3.22 SLIM source code development interface in FD on Desk.....	44
Figure 3.23 FTP transfer window displaying robot directories.....	44
Figure 3.24 Teach Pendant compilation interface screen.....	45
Figure 4.1 System hardware.....	46
Figure 4.2 RMSE calculation	47
Figure 4.3 Calculate angular error	47
Figure 4.4 Image batch under vision analysis	47
Figure 4.5 Database query log showing cycle times and latency records.....	48

LIST OF TABLE

Table 1.1 Comparative Analysis of Existing Part Feeding and Sorting Solutions.	4
Table 1.2 Parameter table	5
Table 2.1 Workspace parameters	16
Table 2.2 Technical specifications of the KEYENCE KV-5500 PLC.....	18
Table 2.3 Technical specifications of the HG-KR23B servo motor	20
Table 2.4 Technical specifications of the MR-J4-20A servo drive.....	20
Table 2.5 Technical specifications of the OMRON S8VK-S03024 power supply.....	21

ABBREVIATIONS

PCA	Principal Component Analysis
IIoT	Industrial Internet of Things
HTTP	HyperText Transfer Protocol
ROI	Region Of Interest
GPU	Graphics Processing Unit
PLC	Programmable Logic Controller
DM	Data Memory
REQ	Request
ACK	Acknowledgement
AI	Artificial Intelligence
PC	Personal Computer
SLIM	Standard Language for Industrial Manipulator
CFD	Computational Fluid Dynamics
RMSE	Root Mean Square Error

CHAPTER 1: CHALLENGES AND SOLUTIONS FOR AUTOMATED BOLT SORTING

1.1 Project overview

1.1.1 Problem statement

In the context of Industry 4.0, manufacturing systems are rapidly evolving toward automation and flexible production. In mechanical assembly industries, components such as bolts, and other small metallic parts are often distributed randomly in mixed conditions, creating challenges for automated robotic picking systems. Traditional feeding systems are difficult to adapt to product variations, as previous studies have stated that “traditional vibratory feeders are inflexible towards part variations” [2]. Therefore, flexible feeding solutions are increasingly required in modern manufacturing environments.

To enable accurate robotic manipulation, "components need to be presented to the robot in a known position and orientation" [1]. This means the system must not only detect the target object but also determine its position, orientation, and grasping coordinates before robot execution. In many industrial environments, robots, cameras, and PLCs are already available but have not yet been integrated into a complete flexible feeding system [3]. Hence, this research focuses on integrating machine vision, Flex Feeder technology, and robotic manipulation to identify and pick target bolts from a cluttered distribution containing various other mixed objects in real time.

1.1.2 Overview of existing solutions

To address the problem of automated part sorting in industrial environments, several technological approaches have been developed and widely adopted. Depending on the application requirements, these solutions focus on different aspects of the sorting process, including part feeding, part separation, object recognition, and classification. Among the most common approaches are traditional vibratory feeding systems, conveyor-based sorting systems, and machine vision-based recognition systems. Each solution provides specific advantages but also exhibits limitations when applied to cluttered environments containing randomly distributed and overlapping components.

First, Traditional Feeding System – Vibratory Bowl Feeder



Figure 1.1 Vibratory bowl feeder for automatic part feeding.

Figure 1.1 illustrates a vibratory bowl feeder, one of the most widely adopted solutions for automatic part feeding in industrial production. The system utilizes mechanical vibration to orient and transport components into predefined positions, enabling continuous and high-speed feeding operations. Due to its reliability and high throughput, the vibratory bowl feeder has been extensively applied in mass production environments [2]. However, the primary limitation of this approach is its lack of flexibility. Each feeder is typically designed and optimized for a specific component geometry, making it difficult and costly to adapt when product dimensions or shapes change. Consequently, vibratory bowl feeders are less suitable for modern manufacturing systems that require frequent product variations and flexible production capabilities [2].

Second, Conveyor-Based Part Sorting System

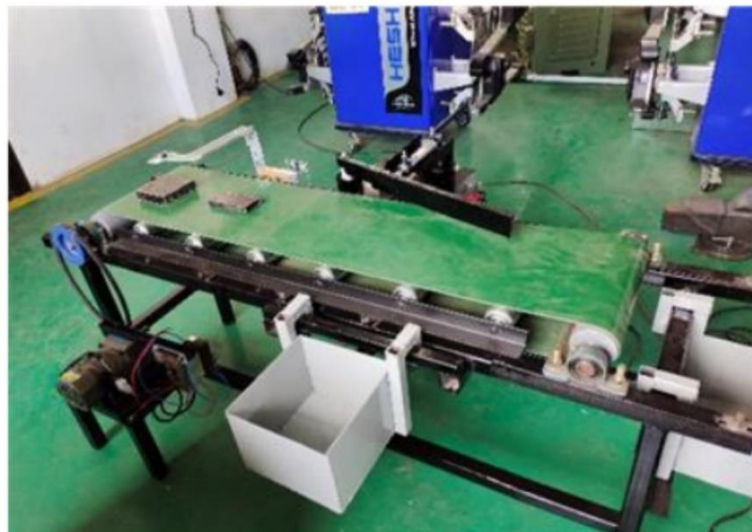


Figure 1.2 Conveyor-Based part sorting system.

Figure 1.2 presents a conveyor-based automated sorting system that employs sensors and pneumatic actuators to classify products during transportation. Components are conveyed along a predefined path while sensors detect specific physical characteristics

and trigger the sorting mechanism accordingly. This approach offers high sorting efficiency, reduced labor requirements, and reliable operation in structured industrial environments [3]. Nevertheless, conveyor-based systems are generally limited to classification based on simple physical characteristics such as size, height, or presence detection. Their performance depends heavily on proper part presentation and sensor calibration. Furthermore, these systems are not well suited for handling randomly distributed or overlapping components, which are common challenges in industrial feeding applications [3].

Finally, Machine Vision Solutions



Figure 1.3 Industrial machine vision system for automated inspection.

Figure 1.3 shows a typical industrial machine vision system used for automated inspection and object recognition. Machine vision systems employ cameras and image processing algorithms to identify components based on geometric, dimensional, or visual features. This technology enables the recognition of multiple object categories within a single platform and provides position and orientation information for subsequent automation tasks [4]. However, machine vision systems alone cannot perform physical manipulation or separation of objects. In addition, recognition performance may degrade under challenging conditions such as uneven illumination, metallic reflections, occlusions, and overlapping components. Therefore, although machine vision provides valuable object information, it typically requires integration with additional handling mechanisms to achieve a complete automated sorting solution [3].

1.1.3 Compare the solutions

Table 1.1 Comparative analysis of existing part feeding and sorting solutions.

Solution	Main Characteristics	Advantages	Limitations
Vibratory Bowl Feeder	Uses mechanical vibration to orient and transport parts into predefined positions.	High throughput, stable operation, and widely used in mass production environments.	Designed for specific component geometries, difficult to adapt to product changes, and lacks flexibility for handling multiple bolt types.
Conveyor-Based Part Sorting System	Uses conveyor transportation combined with sensors and actuators for automatic classification.	Reduces manual labor, provides reliable operation, and achieves efficient sorting in structured environments.	Sorting is generally limited to simple physical attributes such as size or height. Performance depends on proper part presentation and sensor calibration.
Machine Vision System	Utilizes cameras and image processing algorithms to recognize and localize objects.	Capable of recognizing multiple object types and determining object position and orientation.	Sensitive to lighting variations, reflections, and occlusions. Cannot physically separate or manipulate objects without additional handling mechanisms.

From reviewing current industrial part feeding and picking technologies, it is apparent that no single standalone technology can simultaneously satisfy the increasing demands for flexibility, accuracy, and adaptability in modern production environments. Traditional feeding systems can achieve high throughput but are generally limited when handling a wide variety of parts. Similarly, independent machine vision systems can effectively recognize objects, yet their performance depends significantly on the distribution state and the degree of separation of parts within the field of view. To overcome these limitations, modern automation trends are shifting toward robotic guidance systems in which the feeding mechanism, machine vision system, and industrial robot are integrated into a single unified system.

1.2 Proposed system

1.2.1 Proposed solution

Therefore, this research focuses on developing a robot guidance system comprising a flexible feeder (Flex Feeder), a deep-learning-based machine vision system utilizing the YOLOv8 Segmentation model, and a 6-degree-of-freedom Nachi MZ04 industrial robot. In this architecture, the feeder randomly distributes parts within the camera's

field of view, the machine vision system performs object segmentation and locates each part, while the robot utilizes the provided coordinates to perform the pick-and-place operations. Through this integration, the system is expected to achieve the following objectives:

- High flexibility in handling multiple types of bolts and mechanical parts with varying shapes without modifying the mechanical structure of the feeding system.
- High accuracy during object detection, individual instance segmentation, localization, and picking operations under complex conditions such as light reflections, occlusions, or overlapping parts.
- Stable synchronization among the feeder, machine vision system, and robot controller to guarantee continuous and reliable operation.
- Practical applicability in industrial environments by reducing human intervention, enhancing manufacturing line flexibility, and optimizing the overall system cycle time.

1.2.2 Geometric and surface characteristics of the research object

In automated manufacturing and assembly systems, bolts are common mechanical components but frequently appear in a cluttered distribution with highly similar dimensions. Within the scope of this thesis, the M8×15 mm bolt is selected as the target object to demonstrate the flexible sorting capability of the proposed system. The objective is to accurately identify and grasp the M8×15 mm bolts from a mixed environment containing various other bolts, thereby evaluating the system's sorting performance and scalability. Alongside the target object, multiple types of bolts and mechanical components are arranged as distractor objects to create a complex feeding environment similar to actual production conditions.

1.2.2.1 Geometric characteristics

The target bolts may appear in different spatial states on the feeder tray, including standing upright, lying on their sides, or rotating in various directions. In addition, overlapping frequently occurs when target objects partially cover one another or are partially occluded by surrounding components. These geometric characteristics increase the complexity of object localization and pose estimation. Therefore, the machine vision system must not only distinguish the target bolts from distractor objects but also accurately determine the center coordinates (X, Y) and rotational orientation angle (θ) of each valid M8×15 mm bolt for robotic grasping.

Table 1.2 Parameter table

No.	Bolt name	Thread Pitch P(mm)	Tap Drill Diameter (D)	Length (L)	Role
1	M8x15 mm	1.25	6.8	15	Target Object

1.2.2.2 Surface characteristics

In terms of material, the majority of the components are manufactured from carbon steel with galvanized surface treatment. This characteristic generates high and uneven light reflectance, which can easily cause local specular reflections under conventional illumination conditions. The resulting glare may alter the apparent contour and mask boundaries of the bolts, particularly around edges and corners. Variations in image appearance caused by these reflections represent a typical challenge for machine vision systems, requiring robust recognition algorithms capable of accurately distinguishing M8×15 mm target objects from other miscellaneous components under varying lighting conditions.

1.2.3 Hierarchical architecture and overall block diagram

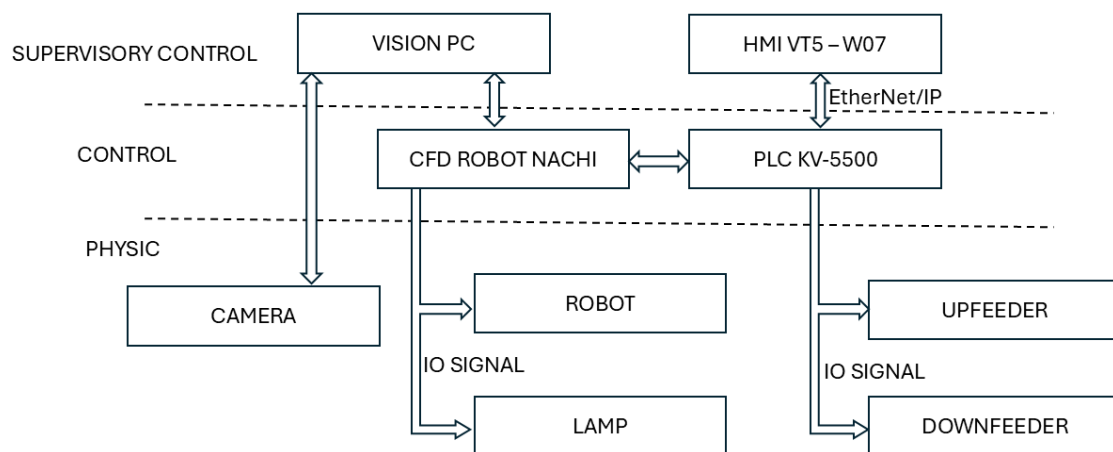


Figure 1.4 Layered functional architecture of the system.

The system is organized according to a three-layer functional architecture, consisting of the supervisory layer, control layer, and field layer. The Vision PC is responsible for image processing and provides recognition data; the PLC controls the part-feeding mechanism; and the CFD robot controller acts as the central unit coordinating the operating sequence. Based on the recognition results from the Vision PC, the robot determines the picking actions and sends the necessary control signals to the PLC to perform the part spreading, part flipping, or purge modes. This architecture clearly separates the functional blocks while still ensuring coordinated operation throughout the entire system.

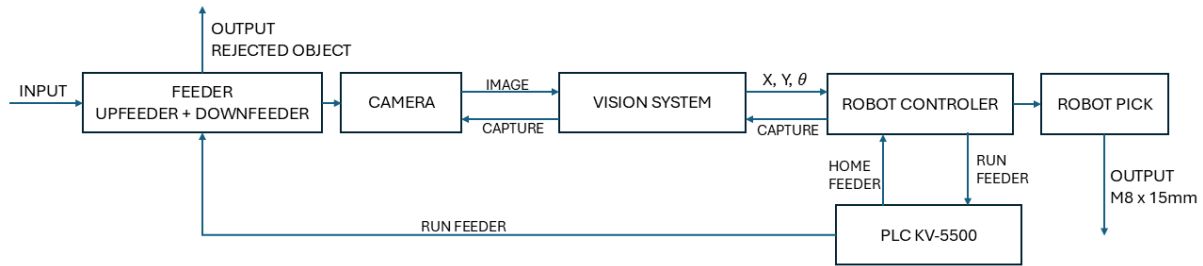


Figure 1.5 Overall block diagram.

Figure illustrates the overall architecture of the proposed vision-guided robotic sorting system. The Flex Feeder, consisting of the Up Feeder and Down Feeder, distributes workpieces within the camera's field of view. The industrial camera captures images of the feeder workspace and transmits them to the Vision System for processing. The Vision System applies object recognition and localization algorithms to determine the position coordinates (X, Y) and orientation angle (θ) of valid target objects. This information is then sent to the Robot Controller, which generates the corresponding pick-and-place motions for the robot. The PLC KV-5500 controls the feeder operation based on commands received from the robot controller, including spreading, flipping, and purge functions. Through the coordination of these components, the system achieves automatic detection, localization, and sorting of M8×15 mm bolts in a continuous operating cycle.

1.3 Chapter summary

This chapter has introduced the problem of automated bolt sorting in industrial environments and highlighted the challenges caused by random distribution, overlapping parts, and reflective surfaces. Several existing solutions, including traditional feeding systems, flexible feeding systems, machine vision solutions, and vision-guided robotic systems, were reviewed and compared in terms of flexibility, accuracy, and integration complexity. Based on this analysis, the proposed system was presented as an integrated solution combining a Flex Feeder, a machine vision system, and a Nachi MZ04 robot. In addition, the geometric and surface characteristics of the M8×15 mm bolt were identified as the basis for the system design. The next chapter will present the theoretical foundation and related components used in the proposed system.

CHAPTER 2: THEORETICAL FOUNDATION AND RELATED COMPONENTS

2.1 System requirements specification

2.1.1 Spatial positioning accuracy

The spatial positioning accuracy refers to the ability of the vision–robot system to estimate and reach the planar coordinates (X,Y) of a workpiece on the vibrating tray within an acceptable tolerance.

In this study, the accuracy is evaluated using the root mean square error (RMSE), with a target requirement of:

$$\text{RMSE} \leq 5 \text{ mm}$$

This threshold is derived from the system error budget. The robotic manipulator (Nachi MZ04 robot arm) provides a repeatability of approximately 0.02 mm [6], while the remaining error is mainly caused by image noise and homography transformation inaccuracies.

Maintaining the RMSE within 5 mm ensures sufficient alignment between the gripper and the object, thereby reducing the likelihood of misgrasping.

2.1.2 Rotational orientation accuracy

Rotational orientation accuracy refers to the ability of the vision system to estimate the inclination angle θ of the workpiece, enabling proper alignment of the robot end-effector (J6 axis) during grasping

Propose value: Angular error

$$\Delta\theta < 5^\circ$$

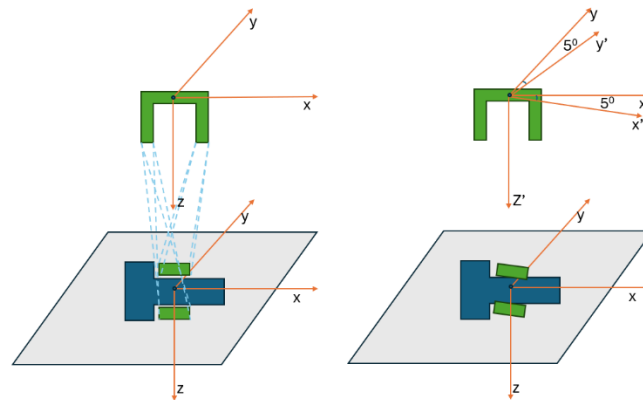


Figure 2. 1 Illustration of gripper orientation alignment and angular error during picking

The orientation is extracted using Principal Component Analysis (PCA) on the segmented object. This threshold ensures that the gripper jaws remain approximately parallel to the longitudinal axis of the workpiece, thereby maximizing contact stability. Angular errors exceeding 5° may result in eccentric gripping and increase the risk of slippage during high-speed robot motion.

2.1.3 Reliability & pick success rate

System reliability is evaluated based on the ability of the robot to successfully pick, hold, and release the target workpieces identified by the vision system without failure. Expected value:

$$\text{Pick success Rate} \geq 90\%$$

To improve the overall picking success rate, the system implements a re-detection strategy. When a detected object is not selected for coordinate transmission from the Vision PC to the robot, the feeder redistributes the remaining objects to reduce overlap and occlusion. A new image is then acquired, and the object detection process is executed again. The newly detected objects are re-evaluated and added to the picking queue for subsequent pick-and-place operations.

2.2 Theoretical foundation

The proposed vision-guided sorting system combines deep learning, geometric analysis, and coordinate transformation techniques to enable accurate robotic picking of target bolts. First, images acquired from the industrial camera are processed by the YOLOv8 Segmentation model to identify target objects and generate pixel-level segmentation masks. The segmentation results are then filtered based on geometric characteristics to remove invalid detections and overlapping objects.

Subsequently, the Principal Component Analysis (PCA) algorithm is applied to the extracted mask to estimate the orientation angle of each valid bolt. The centroid coordinates obtained from the segmentation mask are transformed from image coordinates into real-world coordinates through a Homography calibration matrix. Finally, the calculated position and orientation information are transmitted to the robot controller, allowing the robot to perform accurate pick-and-place operations. This integrated framework combines the robustness of deep learning-based recognition with geometric post-processing techniques to achieve reliable operation in industrial environments.

2.2.1 Theoretical basis of image processing and deep learning

In this system, OpenCV is used as a classical image processing layer to clean input images, perform background separation, binarize, extract contours, and measure object geometry. The execution pipeline follows the sequence:

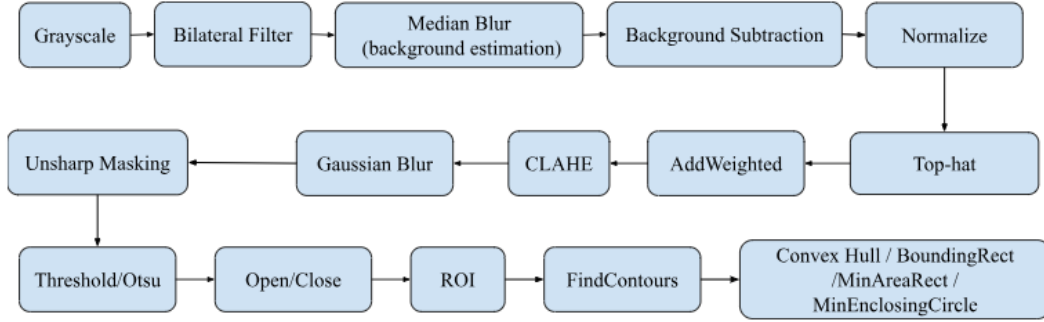


Figure 2. 2 Execution process

Subsequently, geometric features are applied toward size filtering, robot safety, and synchronization with the working coordinate system.

In this system, OpenCV is employed as a classical image-processing layer to prepare the acquired image for subsequent geometric analysis and object localization. First, the Neptune camera image is first converted from the BGR color space into grayscale using `Cv2.CvtColor(frame, gray, ColorConversionCodes.BGR2GRAY)`. The grayscale intensity is computed as:

$$Y = 0.299R + 0.587G + 0.114B \quad (2.1)$$

where R, G, and B denote the intensities of the three color channels, and Y represents the equivalent brightness value. This conversion reduces the image data from three channels to a single channel, thereby improving the stability of subsequent thresholding, morphology, and contour extraction operations. After conversion to grayscale, the image is filtered using a bilateral filter [15], [16]:

$$I'_{(p)} = \frac{1}{W_p} \sum_{q \in \Omega} I_{(q)} G_s(\|p - q\|) G_r(|I_{(p)} - I_{(q)}|) \quad (2.2)$$

With:

$$W_p = \sum_{q \in \Omega} G_s(\|p - q\|) G_r(|I_{(p)} - I_{(q)}|) \quad (2.3)$$

where p is the pixel under consideration, q is a neighboring pixel, Ω denotes the filtering window, G_s is the spatial distance weight, and G_r is the intensity difference weight. Unlike conventional Gaussian filtering, the bilateral filter incorporates both spatial proximity and radiometric similarity, enabling effective noise suppression while preserving structural boundaries and minimizing edge blurring caused by metallic reflections.

Next, the image passes through a median blur [17]:

$$I_{med}(x, y) = \text{median}\{I(i, j)\} \quad (2.4)$$

With: $(i, j) \in N_k(x, y)$

where $N_k(x, y)$ denotes a neighborhood of size $k \times k$. In contrast to an arithmetic mean filter, the median filter preserves edges while eliminating impulse noise. In this application, a large median kernel is primarily used to estimate a slowly varying local background, generating a background image for the subsequent background subtraction stage. This compensation is essential because the vibratory tray surface and metallic reflections often exhibit non-uniform intensity distributions. After background subtraction and normalization, the background-compensated image is subjected to a top-hat operation [17], [18]:

$$\text{TopHat}(I) = I - \text{Opening}(I) \quad (2.5)$$

With: $\text{Opening}(I) = \text{Dilation}(\text{Erosion}(I))$

where: I : input image

Erosion: erosion operation (shrinks bright regions)

Dilation: dilation operation (restores regions remaining post-erosion)

The top-hat transform isolates bright features that are smaller than the local background structure, making it particularly suitable for extracting bolt boundaries and reflective regions that appear as small bright patterns on an uneven background. The processed image is subsequently refined through AddWeighted, CLAHE, Gaussian Blur, and Unsharp Masking before the final binarization stage. Following pre-processing, the image is binarized as:

$$B(x, y) = \begin{cases} 255, & I(x, y) > T \\ 0, & I(x, y) \leq T \end{cases} \quad (2.6)$$

where, under BinaryInv mode, the output levels 0 and 255 are inverted. If the manual threshold parameter is set to 0, the application switches to Otsu's thresholding method [19], [20]:

$$T^* = \arg \max_T \sigma_b^2(T) \quad (2.7)$$

Where:

$$\sigma_b^2(T) = w_0(T)w_1(T)(\mu_0(T) - \mu_1(T))^2 \quad (2.8)$$

w_0, w_1 are the probabilities of the background and object classes, while μ_0, μ_1 represent the mean intensities of the respective classes. Otsu's method automatically determines the optimal threshold by maximizing the between-class variance, thereby making the two pixel groups as distinct as possible. In the implemented application, fixed thresholding is adopted when illumination conditions remain stable, whereas Otsu's method is employed when lighting varies but the image histogram still exhibits two relatively distinct peaks.

2.2.2 YOLOv8 Segmentation algorithm YOLOv8

Segmentation is a deep learning model applied to object detection and instance segmentation tasks within images. Unlike standard object detection methods that only provide a bounding box, the segmentation model allows precise delineation of object boundary profiles on the input image.

In this project, YOLOv8 Segmentation is utilized to recognize target bolts in a cluttered feeding environment. The output results of the model include the object's position in the image, classification labels, segmentation masks, and prediction confidence scores. From the segmentation outputs, the system proceeds with processing to determine the center point and orientation angle of the bolt, which aids in the interpolation of coordinates for robotic grasping. YOLOv8 belongs to the class of one-stage object detection models, enabling direct inference on the input image with high processing speeds, matching the demands of real-time industrial machine vision applications. The segmentation mask region is subsequently utilized as input for the PCA algorithm to establish the bolt's orientation angle[6].

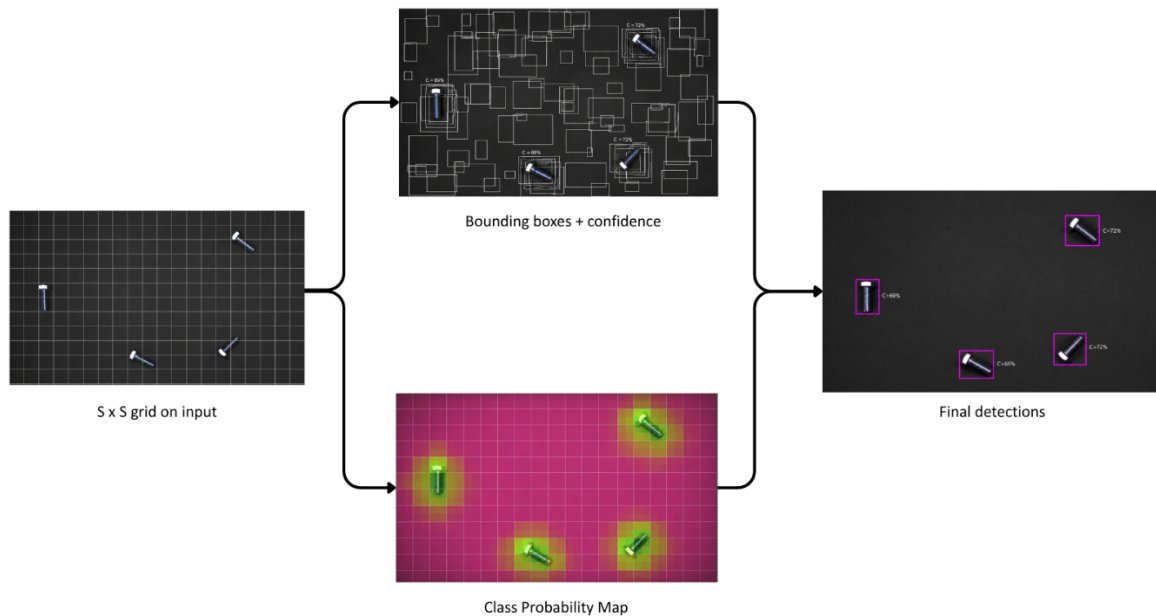


Figure 2. 3 YOLO algorithm operations

YOLO features a foundational architecture composed of two main sections: base networks and extra layers. The base networks are Convolutional Neural Networks (CNNs) pre-trained on the COCO dataset, which are commonly utilized as backbones in object detection applications.

The extra layers are designated for analyzing features and detecting objects, containing layers such as: Conv, C2f, SPPF, and Detect.

YOLO utilizes a squared error function based on predictions and ground truth labels to compute model loss. Specifically, the total loss is the summation of the following three sub-losses:

- Classification loss: Error in predicting the object class label.
- Localization loss: Error in predicting the coordinates, length, and width of the boundary box.
- Confidence loss: Error regarding whether a grid cell contains an object.

Following this, a new rectangle that completely encloses the mask with the minimum possible area is found using the MinAreaRect algorithm from the OpenCV library

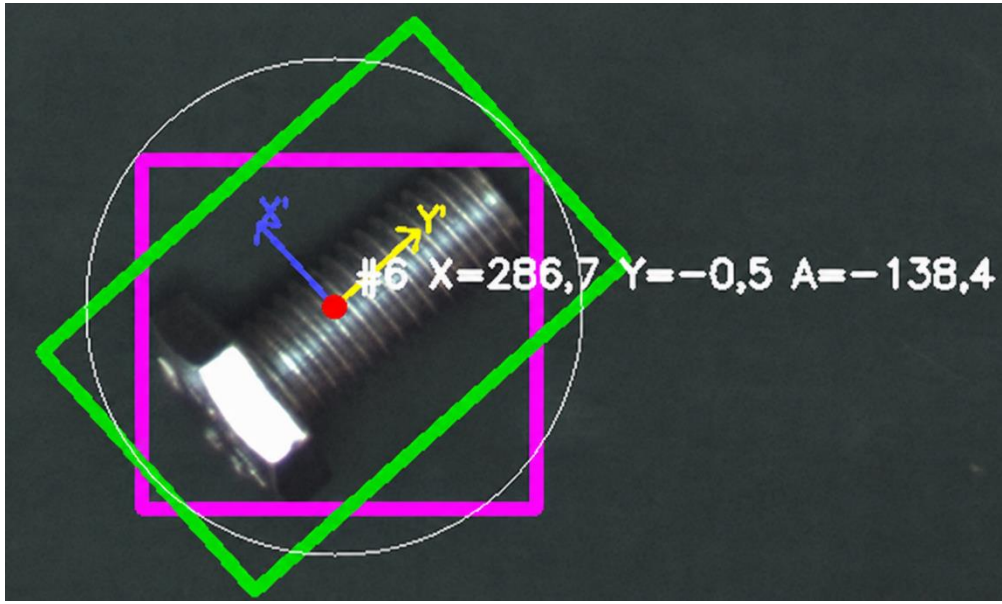


Figure 2. 4 Result of the MinAreaRect Algorithm

As observed from the figure, the pink bounding box represents the bounding box acquired directly after running Segmentation, while the green bounding box is the output of the MinAreaRect algorithm. From this point, the angle is computed via the angle between the long edge vector of the green bounding box and a horizontal reference axis vector of the image. This provides the initial proposed angle for the component.

2.2.3 PCA Algorithm for determining rotation angle

After obtaining the binary mask of the target object, the Principal Component Analysis (PCA) algorithm is applied to determine the primary orientation of the object in 2D space [7].

The mathematical procedure includes:

Centroid Determination: The pixel centroid coordinates (x_c , y_c) are calculated based on the spatial moments of the image mask $I(x,y)$:

$$x_c = \frac{\sum \sum x \cdot I(x,y)}{\sum \sum I(x,y)}, \quad y_c = \frac{\sum \sum y \cdot I(x,y)}{\sum \sum I(x,y)} \quad (2.9)$$

Covariance Matrix Construction: To evaluate the geometric dispersion of the object around the determined centroid, the system establishes a covariance matrix C :

$$C = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} \\ \sigma_{xy} & \sigma_{yy} \end{bmatrix} \quad (2.10)$$

The components within the matrix are calculated in detail using the squared distance method:

$$\text{Variance along the X-axis: } \sigma_{xx} = \frac{\sum \sum (x-x_c)^2 \cdot I(x,y)}{\sum \sum I(x,y)} \quad (2.11)$$

$$\text{Variance along the Y-axis: } \sigma_{yy} = \frac{\sum \sum (y-y_c)^2 \cdot I(x,y)}{\sum \sum I(x,y)} \quad (2.12)$$

$$\text{X-Y Covariance (Cross-correlation): } \sigma_{xy} = \frac{\sum \sum (x-x_c)(y-y_c) \cdot I(x,y)}{\sum \sum I(x,y)} \quad (2.13)$$

Eigenvector Extraction and Orientation Angle θ Calculation: The system solves the characteristic equation of the covariance matrix C to find the corresponding eigenvalues and eigenvectors.

The eigenvector corresponding to the largest eigenvalue defines the direction of the Major Axis, which is the most extended distribution direction of the bolt body.

Where: $v_{1,x}$ is the projection component of the vector v_1 onto the horizontal axis (x) of the image space.

$v_{1,y}$ is the projection component of the vector v_1 onto the vertical axis (y) of the image space.

From these spatial components, the orientation angle θ of the object is extracted via the trigonometric function:

$$\theta = \arctan2(v_{1,x}, v_{1,y}) \quad (2.14)$$

The centroid coordinates (x_c, y_c) are then multiplied by the Homography calibration matrix to convert them into the real world coordinate system (X, Y) (in mm).

2.2.4 Theoretical basis of coordinate calibration

In vision-guided robotic cells, establishing a spatial transformation mapping between the 2D pixel coordinate system of the camera and the 3D physical coordinate system of the robot arm is mandatory. Within the scope of this system, since the industrial camera is rigidly positioned so that its image plane is strictly parallel to the flat surface of the Flex Feeder tray, the depth parameter (Z) along the optical axis can be treated as a constant. Therefore, the transformation can omit the Z dimension, effectively reducing the number of variables in the transformation matrix. By deploying the Homography method, a 3 x 3 transformation matrix H is defined to project coordinates from the image frame to the robot workspace:

$$\begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & t_{13} \\ t_{13} & r_{22} & t_{23} \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_{pixel} \\ y_{pixel} \\ 1 \end{bmatrix} \quad (2.15)$$

The unknown transformation coefficients ($r_{11}, r_{12}, t_{13}, r_{21}, r_{22}, t_{23}$) are computed using linear regression or the Least Squares method in linear algebra. This mathematical approach minimizes the squared residuals between the measured points, providing an optimal transformation model for physical plane-to-plane calibration.

2.2.5 Communication interface theory

The system employs the TCP Socket protocol over an Ethernet backbone to exchange data between the Vision PC and the Nachi robot controller. After completing image processing and establishing object positions, the Vision PC transmits coordinate data to the robot across the TCP connection. The robot controller acquires the data, converts it into working coordinates, and executes the picking and placing operations. The TCP

protocol was selected to guarantee stability and data transmission reliability within an industrial environment[5].

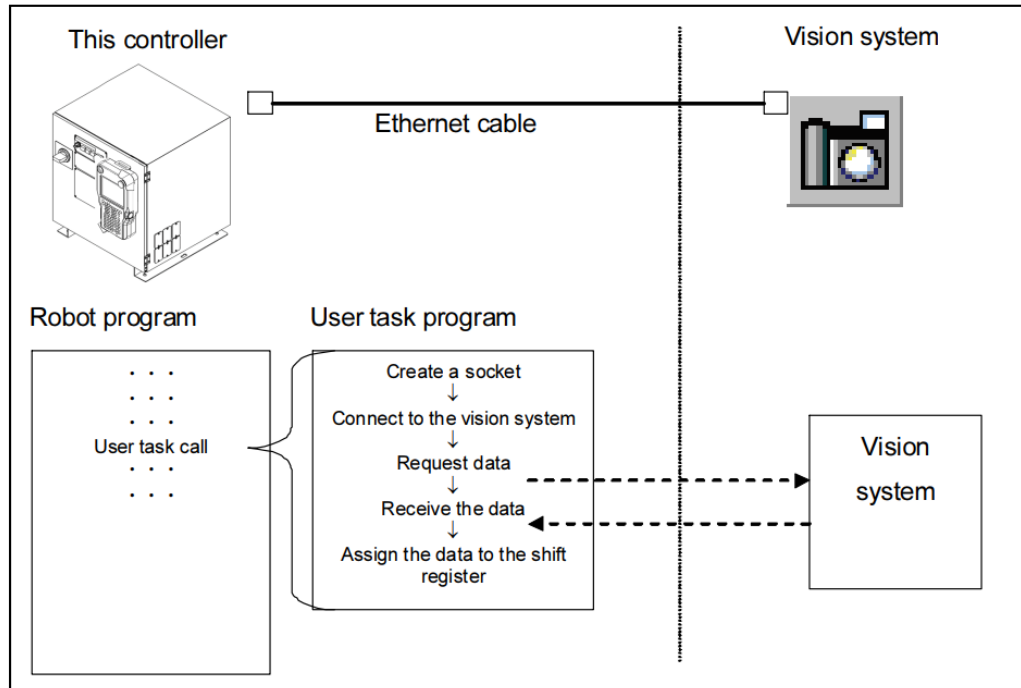


Figure 2.5 TCP Socket interface concept between controller and vision system

Flask Framework: Flask is a micro web framework written in Python. It is simple to use for creating API endpoints to deploy image processing algorithms via HTTP protocols without needing third-party tools or heavy libraries. However, running a standalone Flask instance operates locally on the host machine.

2.3 Related equipment

2.3.1 Nachi MZ04 robot

The Nachi MZ04 is a high-speed 6-axis industrial robotic arm designed to accommodate strict space constraints and meet high-performance demands in small part sorting and classification setup [5]. Regarding basic technical parameters, the robot features a maximum payload capacity of 4 kg and a working radius extending up to 541 mm, allowing it to seamlessly cover the entire operating footprint of the Flex Feeder vibrating tray [5]. A key structural advantage of this model is its hollow wrist design; by routing gripper cabling and pneumatic tubes directly through the robot wrist, the system completely eliminates cable interference during complex, high-speed rotational movements, thereby significantly enhancing wire harness reliability and operational life [5]. Furthermore, the robot achieves a positioning repeatability of ± 0.02 mm, an essential specification that guarantees precision during pick-and-place actions, even when handling small-scale fasteners or navigating tight spaces within the system [5].

In this system, the robot executes the following functions:

- Receives coordinate data from the vision software.

- Moves to the designated pick positions.
- Adjusts the J6 axis angle according to the target's orientation angle.
- Grasps the target component and places it into a pre-assigned location.
- Transmits cycle-completion signals or request purge commands.

Thanks to its high speed and stable accuracy, the Nachi MZ04 effectively fulfills the demands of the flexible feeding system.

Its broad operational envelope is structured across the following workspace parameters:

Table 2.1 *Workspace parameters*

Motor	Workspace
Axis 1	$-170^{\circ} \div 170^{\circ}$
Axis 2	$-55^{\circ} \div 180^{\circ}$
Axis 3	$-70^{\circ} \div 190^{\circ}$
Axis 4	$-190^{\circ} \div 190^{\circ}$
Axis 5	$-120^{\circ} \div 120^{\circ}$
Axis 6	$-360^{\circ} \div 360^{\circ}$

Machine Coordinate System:

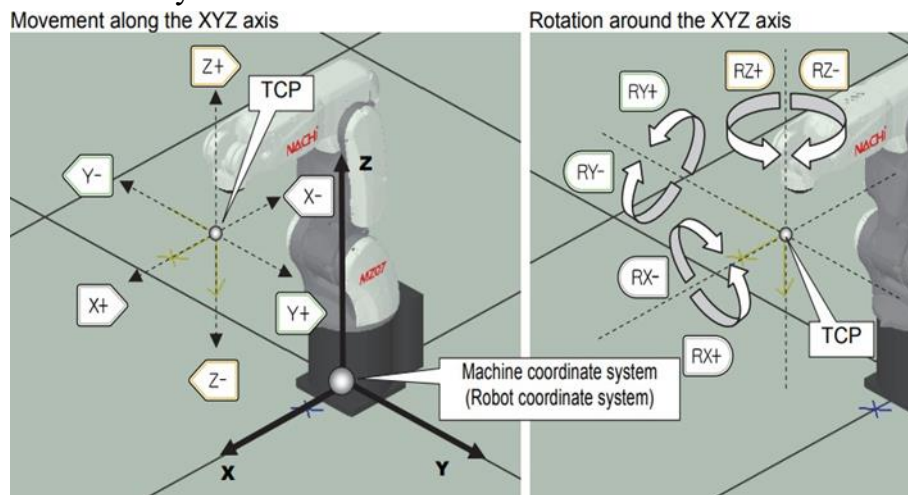


Figure 2. 6 Nachi Robot coordinate systems

In this coordinate system, the Tool Center Point (TCP) of the robot must move based on a fixed coordinate system as shown in the image above, known as the Machine Coordinate System (Robot Coordinate System). Jog operations with the X, Y, and Z buttons control the tool tip to move along the corresponding robot axes, and operations with RX, RY, and RZ control the tool tip to rotate around a designated fixed axis. Because it must maintain synchronization with the robot axes, the interpolation or movement speed of the robot within this coordinate system is slower compared to when it operates in the Joint Coordinate System.

Tool Coordinate System:

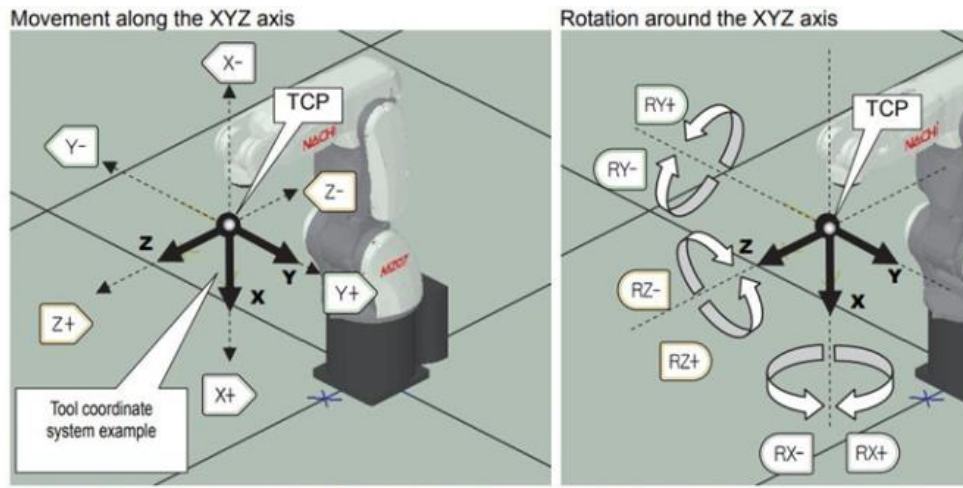


Figure 2. 7 Tool coordinate system

Similar to the Robot Coordinate System, but instead of synchronizing movements along the main structural axes of the robot, it moves synchronously according to the configured coordinate axes of the TCP. In normal situations, users tend to align both the Robot and Tool coordinate frames to match, making the two coordinate spaces nearly identical. The objective of controlling the robot to pick up a part is to align the Tool Coordinate System so it coincides with the coordinate system of the workpiece.

CFD Controller Support: The CFD is a controller designed specifically for Nachi robots, supporting up to 7 axes (6 standard axes and 1 integrated auxiliary axis if required). Integrated within are control routines based on Functions and the SLIM language. Concurrently, the CFD Controller permits communication with PLCs and accepts external data streaming via Sockets.

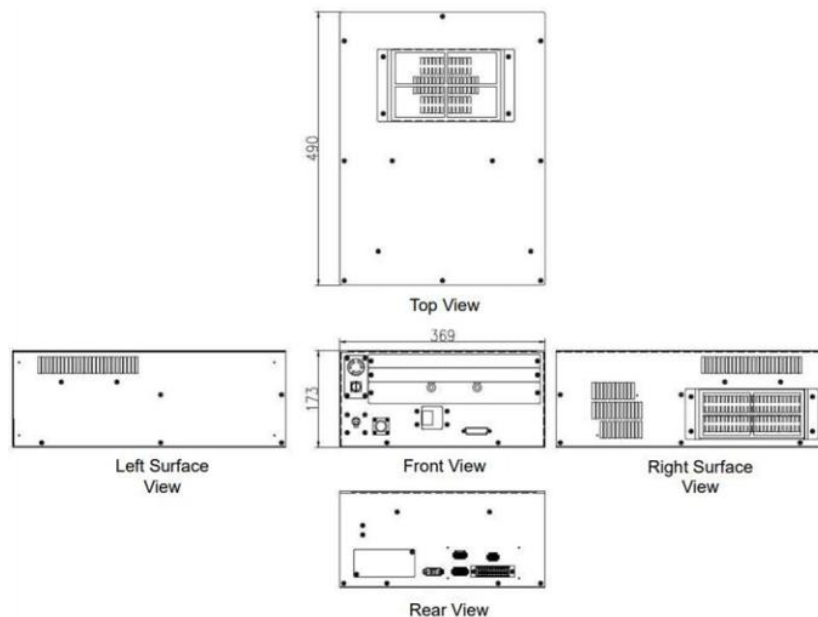


Figure 2. 8 Structural faces of the CFD controller

Nachi robots utilize the specialized programming language SLIM (Standard Language for Industrial Manipulator). Source programs are initiated and edited in ASCII text format, and subsequently compiled into a binary format before being loaded into the

robot controller. This programming method addresses display space limitations and entry efficiency issues on the FD on Desk software screen or Teach Pendant when the system demands complex mathematical calculation algorithms. Because the syntax structure of SLIM shares a strong similarity with Visual Basic aside from specialized motion commands basic control structures such as loops, conditional statements, arrays, and assignment expressions are intuitive, enabling engineers to easily deploy source code.

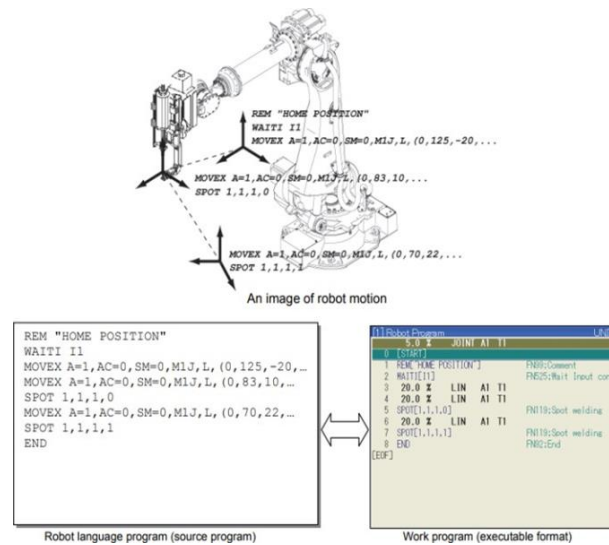


Figure 2. 9 Control programming via ASCII file and executable work program format

2.3.2 PLC control equipment

The Keyence KV-5500 PLC is the primary programmable logic controller utilized in the system. As part of the KV-700 series, its duties involve controlling auxiliary elements, handling input/output signals, synchronizing with the robot, and supporting industrial communication. With fast and accurate information processing capabilities, it aids in optimizing the manufacturing procedure[10].

Table 2.2 Technical specifications of the KEYENCE KV-5500 PLC

Parameter	Value
Power Supply	24VDC ($\pm 10\%$)
Ambient Operating Temperature	0 - 50 °C
Withstand Voltage	1500 VAC
Insulation Resistance	Above 50 MΩ
Current Consumption	320 mA
Maximum Expansion Units	16 modules
Local Built-in I/O	Digital: 16 inputs and 8 outputs; Analog: 2 outputs (Max frequency 100 kHz)
Memory Capacity	260 KB
Bluetooth Support	Yes
EtherNet/IP Support	Yes
Dimensions	72(W) x 90(H) x 85(D) mm
Weight	320g

In addition to the main unit, the Keyence KV-5500 incorporates expansion modules, including:

- Input expansion module KV-C64XC
- Output expansion module KV-C64TD
- Positioning/Pulse-train module KV-MC40V
- Industrial Ethernet Switch EKI-2525I



Figure 2. 10 PLC keyence kv-5500



Figure 2. 11 HMI VT5-W07

2.3.3 HMI VT5-W07

The HMI VT5-W07 touch screen is integrated into the system to establish an intuitive, real-time control and monitoring interface. The device acts as an interactive bridge between operators and the PLC; it supports data status updates, operational parameter configuration, and triggers automatic system fault alarms. Furthermore, the diverse simulation and communication capabilities of the VT5-W07 optimize the program testing stage, ensuring safe system operation and minimizing machinery downtime.

2.3.4 Flex feeder

A servo motor is utilized to drive the Flex Feeder vibrating platform to generate motion adjustable by frequency, amplitude, and vibration duration. Unlike a basic vibratory mechanism, the servo system permits closed-loop control via encoder feedback, which ensures the vibrating table runs stably and adapts to each specific component type.

The servo system comprises: the servo motor, the servo drive, and the feedback encoder. Through this closed-loop feedback mechanism, the system precisely controls the speed, position, and vibration pattern of the feeding tray.

The servo hardware elements used in the system are:

- HG-KR23B: Suitable for controlling the vibration dynamics of the IFEEEDER platform.
- MR-J4-20A: Receives pulse or analog commands from the KV-MC40V module via the KV-5500 PLC to transmit commands adjusting the servo motor's operation. Simultaneously, real-time speed and position feedback signals from the encoder are processed continuously, ensuring operational stability and reliability for the IFEEEDER system.

Table 2.3 Technical specifications of the HG-KR23B servo motor

Parameter	Value
Rated Power	200 W
Supply Voltage	220 V
Rated Speed	3000 r/min
Maximum Speed	6000 r/min
Rated Torque	0.64 Nm
Maximum Torque	2.2 Nm
Encoder Resolution	22 bit
Weight	1,3 kg

*Figure 2. 12 HG-KR23B servo motor**Figure 2. 13 MR-J4-20A servo drive***Table 2.4 Technical specifications of the MR-J4-20A servo drive**

Parameter	Value
Rated Power	200 W
Supply Voltage	220 V single-phase / three-phase
Supported Axes	1
Safety Function Type	STO (IEC/EN 61800-5-2)
Weight	0.8 kg

2.3.5 Industrial lighting and camera system

The IMI TECH IMC3616UC industrial camera is used to capture images of the vibrating platform surface. The CCS TH2-83X75SW illumination light unit, running at 10W and 24V, assists in highlighting object edge contours and mitigating the impact of metallic surface reflections. The pairing of the camera and illumination represents a decisive factor in the overall efficiency of the recognition sequence.



Figure 2. 14 Camera IMC3616UC



Figure 2. 15 Illumination Light TH2-83X75SW

2.3.6 Power supply units and safety equipment

The power supply plays a fundamental role in ensuring stability for the entire system. The control devices, camera, relays, sensors, and illumination units must all be supplied with sufficient, stable power alongside overload protection mechanisms.

The OMRON S8VK-S03024 is selected to run the control devices, satisfying the operational requirements with a safe margin factor.

Table 2.5 Technical specifications of the OMRON S8VK-S03024 power supply

Parameter	Value
Rated Output Current	1.3 A
Maximum Output Current	1.56 A
Rated Input Voltage	100 - 240 VDC
Output Voltage	24 VDC
Overload Protection	Yes, automatic reset
Overvoltage Protection	Yes, shuts down input voltage and restarts input
Power Rating	15 - 480 W
Ambient Temperature	-40 to 70 °C
Service Life	10 years minimum
Weight	250

The system utilizes the following protective components:

- MCB overload protection arrangement: 1 MCB for the control system, 1 MCB for the robot, 1 MCB for the conveyor system, 1 MCB for the vibrating table system, and a common-phase tied MCB.
- Emergency stop buttons and intermediate safety relays to safeguard operators and hardware equipment, which is a mandatory prerequisite in industrial environments.



Figure 2. 16 OMRON S8VK-S03024 power supply



Figure 2. 18 MCB Panasonic CP40-BA 2P



Figure 2. 17 MCB Panasonic 1P

2.3.7 System assembly model

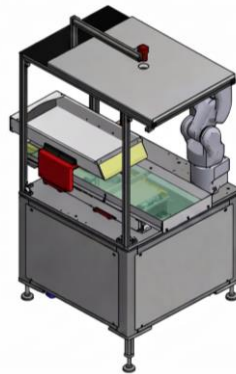


Figure 2. 19 3D CAD Assembly model of the system

2.3.8 KV-Studio software

KV-STUDIO is the programming and management software for Keyence PLCs, offering an integrated development environment for building, monitoring, and debugging control programs. Thanks to its intuitive layout and diagnostic fault tools, it enhances the implementation efficiency of industrial automation projects.

2.3.9 HMI VT-Studio software

VT Studio is specialized software from KEYENCE developed to design and configure the touch-screen interface of the HMI VT Series when linking with PLC systems. This utility supports the entire development cycle, from initiating screen layouts to arranging functional buttons and graphical monitoring charts. Through this, engineers optimize clarity, simplify user operations, and ensure responsive communication speeds with the central PLC.

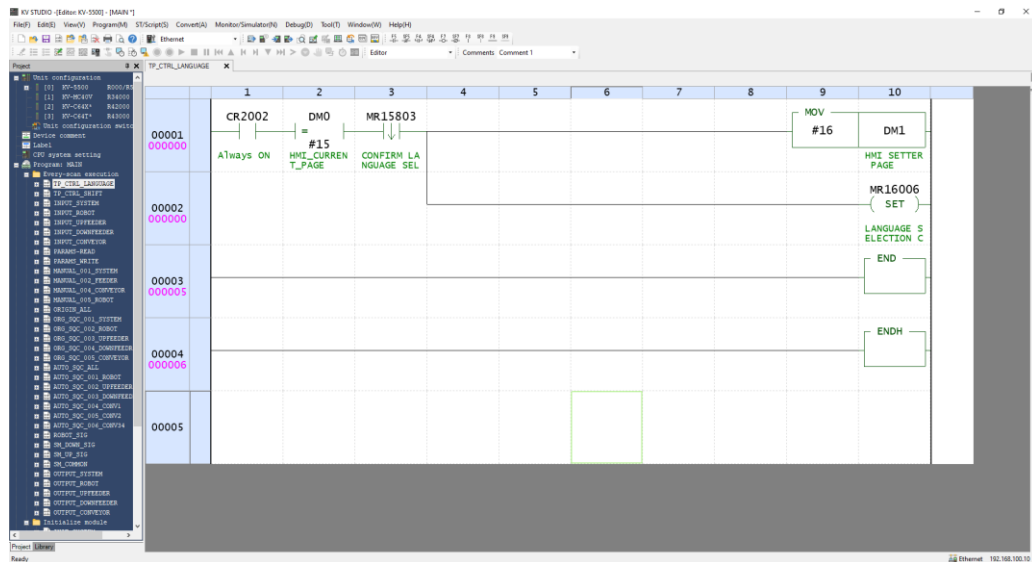


Figure 2. 20 KV Studio software interface showing ladder logic code

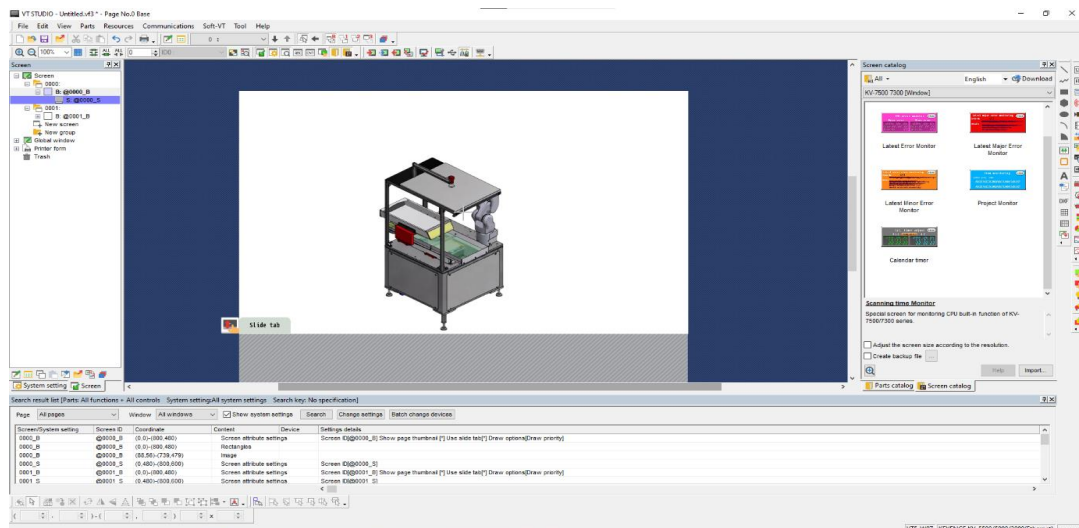


Figure 2. 21 VT Studio software interface showing HMI screen development

2.3.10 FD on Desk software

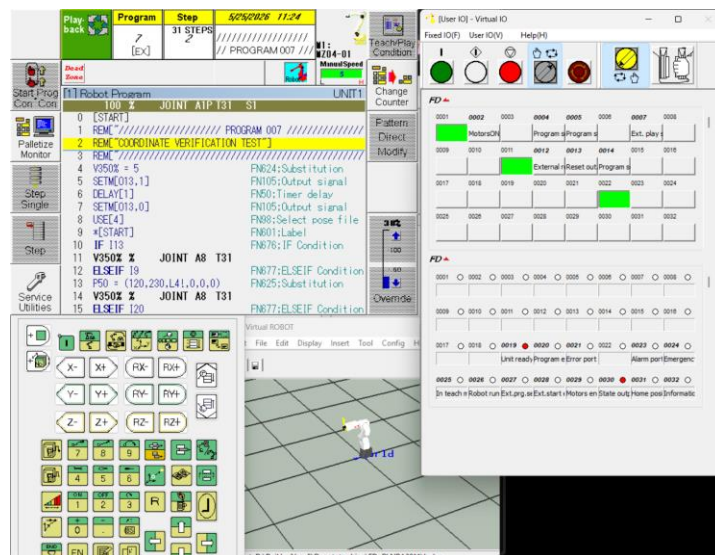


Figure 2. 22 FD on Desk software interface panel

FD on Desk is software created specifically to support programming activities for the CFD robot controller. FD on Desk possesses 3 distinct operating modes:

- View Mode: The color screen of the teach pendant is mirrored onto the PC screen. The computer keyboard can be used to replicate keys on the actual pendant. (However, the robot cannot be jogged manually or recovered from inside this mode). Settings can be adjusted directly through configuration panels, saving edited entries straight onto the robot controller.
- Monitor Mode: Intended for remote monitoring. I/O signals and robot positions of the connected controller can be monitored live on the PC screen. FD on Desk interactions run independently of actual Teach Pendant operations.
- Offline Mode: Enables offline teaching (teaching without a physical robot connected), cycle time simulation, and debugging of the robot language program. It serves well for training personnel without requiring physical hardware and allows completed program files to be uploaded straight into the robot controller.

2.3.11 WinForms (C#) interface development platform

Windows Forms (WinForms) is an open-source graphical class library within the Microsoft .NET Framework, providing a stable foundation to build Human-Machine Interface (HMI) applications running directly on the central control PC. Unlike complex C++ graphics libraries, WinForms operates via an event-driven architecture; the application cycle remains idle until it responds immediately to actions from operators via graphical interaction tools (such as textbox inputs, button presses, or mouse movements).

2.4 Chapter summary

This chapter presented the theoretical background and key technologies used in the automated bolt sorting system, including machine vision, image processing, YOLOv8 Segmentation, PCA orientation estimation, coordinate calibration, and communication methods. The main hardware and software components were also introduced. These foundations serve as the basis for the system implementation and experimental evaluation in the subsequent chapters.

CHAPTER 3. SYSTEM DESIGN AND CONTROL ARCHITECTURE

3.1 Overall system design

The system is designed according to a hierarchical control model where the robot acts as the primary coordinator for the operational cycle. The Vision PC executes image processing and supplies object coordinate data, the PLC controls the Flex Feeder and handles interlocking safety signals, while the robot executes picking and component classification tasks. This organization isolates functional block responsibilities and sets the foundation for synchronized workflow.

Algorithm Flowchart:

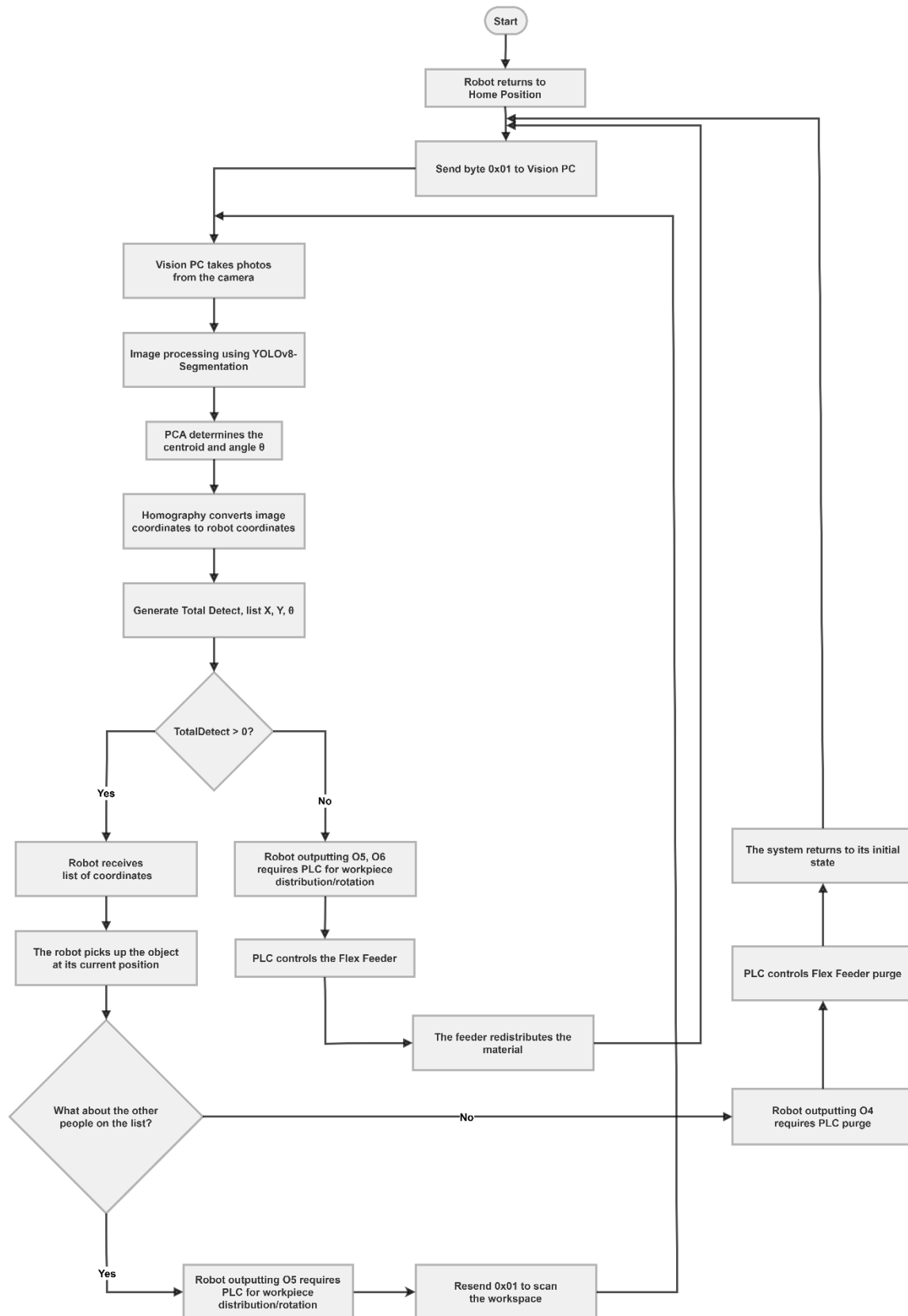


Figure 3.1 Overall system algorithm flowchart

The sorting operation cycle of the system is managed and directed by the Nachi robot controller (acting as the Cycle Master). The operational script is established in a closed-loop sequence via the following collaborative stages:

-
- Stage 1: Initialization and Recognition Triggering: When the system starts in automatic mode (Auto Mode), the robot controller, acting as a Client, opens a TCP/IP Socket connection and sends a standardized command byte with a value of 0x01 to the Flask API server hosted on the Vision PC to request a machine vision task (Detect Request).
 - Stage 2: Image Analysis and Data String Return: Upon receiving the 0x01 signal byte from the robot, the Vision PC immediately triggers the industrial camera to capture an image, passing it through the YOLOv8-Segmentation deep learning core to identify and calculate the gripping coordinate matrix for all valid M8x15mm bolts currently on the tray. These positional arrays (X, Y, theta) are packaged into a JSON data string (coordinate array) and returned to the robot via the HTTP Response protocol of the Flask API.
 - Stage 3: Logic Branching based on Detection Count (Total Detect): Upon receiving the returned data string from the Vision PC, the robot controller checks the total count parameter of detected objects (Total Detect):
 - Empty string case (TotalDetect = 0): Defines that the feeding tray is empty or parts are completely occluded. The robot immediately activates physical digital outputs O5 and O6, transmitting pulse signals to the PLC inputs. The PLC receives this signal and activates corresponding frequency bands to drive the Flex Feeder's servo motor, executing the part-feeding and part-flipping routines. After the configured vibration time finishes, the robot returns to Stage 1, sending byte 0x01 to re-verify the tray surface.
 - Valid part case (TotalDetect > 0): The robot accepts the entire coordinate array and sequentially executes kinematic trajectories, driving the gripper to accurately pick up each component until the transferred coordinate list queue is completely cleared.
 - Stage 4: Cycle Conclusion and Clearance (Purge Mode): Immediately following the final pick command in the data sequence, to prepare for a fresh feeding batch and clear away any rejected parts or residual components, the robot activates physical digital output O4 to the Keyence PLC. The PLC receives the command and immediately activates the purge mode on the Downfeeder tray to eject surplus items out of the workspace area. Once finished, the system loops back to Stage 1.

3.2 Machine vision system design

To develop a robust deep-learning-based vision system capable of instance segmentation, object recognition, and spatial localization of M8×15 mm bolts in a cluttered environment, a YOLOv8-Segmentation workflow was adopted. A total of 5,611 images were collected under different operating conditions to improve model generalization. The image acquisition process included variations in illumination

intensity ($\pm 30\%$), camera viewing angle ($\pm 5^\circ$), and workpiece density ranging from sparse to dense distributions. The dataset was divided into training, validation, and testing subsets with a ratio of 70%, 15%, and 15%, respectively.

All images were annotated using the Roboflow platform with pixel-level instance segmentation masks. During the annotation process, image quality was carefully reviewed to eliminate blurred or incorrectly labeled samples, ensuring consistency throughout the dataset. Figure 3.2 presents the annotation interface used to generate segmentation masks for the target bolts.

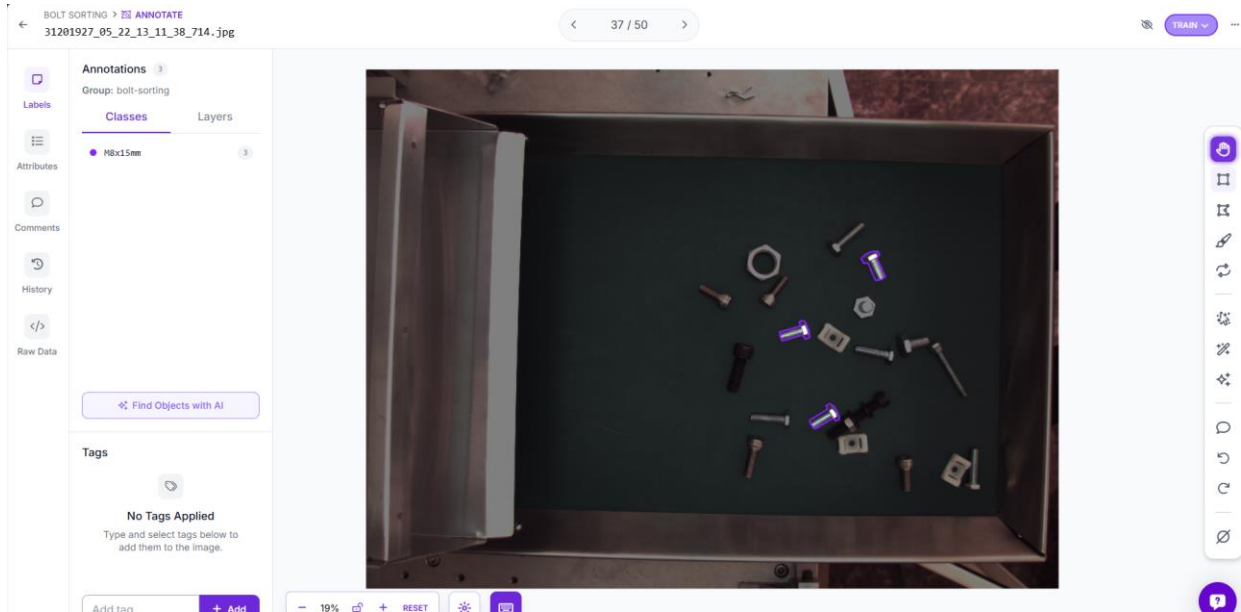


Figure 3.2 Data annotation process interface in Roboflow

To improve the robustness of the trained model, several data augmentation techniques were applied, including horizontal and vertical flipping, random rotation within the range of -45° to $+45^\circ$, brightness variation between -15% and $+15\%$, and cutout augmentation. These transformations increased dataset diversity and enhanced the model's ability to recognize objects under different orientations, lighting conditions, and partial occlusions. The augmentation settings and dataset properties are illustrated in figure 3.3.

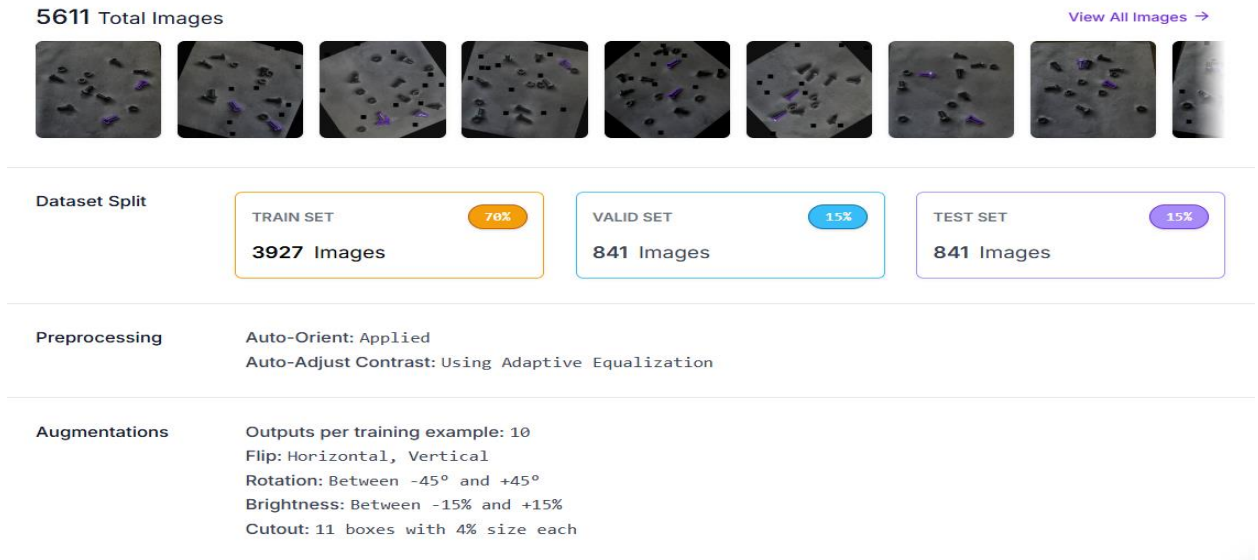


Figure 3.3 Dataset properties and data augmentation configurations

The YOLOv8-Segmentation model was trained using the PyTorch and Ultralytics framework on NVIDIA T4 GPU environments provided by Google Colab and Kaggle. The training configuration utilized a batch size of 16, 300 epochs, a learning rate of 0.001 with learning-rate scheduling, and an input image size of 640×480 pixels. Early stopping based on validation loss and mAP performance was employed to prevent overfitting and ensure stable convergence. Figure 3.4 illustrates the training and validation loss curves obtained during the training process. After training, the optimized model was exported in ONNX format for deployment on the Vision PC, enabling real-time inference in the robotic sorting system.

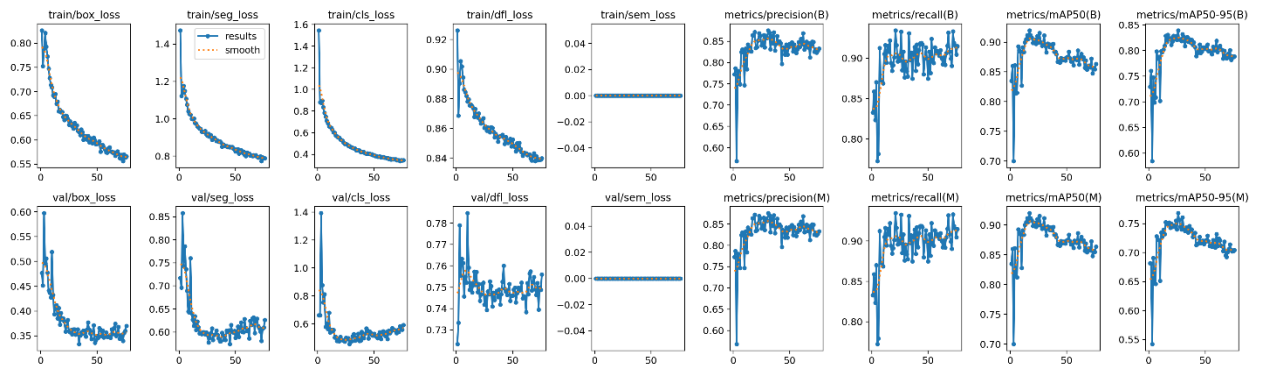


Figure 3.4 A graph illustrating model convergence (Training/Validation loss curves)

The training results shown in Figure 3.4 indicate that the YOLOv8-Segmentation model achieved stable convergence throughout the training process. The training and validation losses for bounding box regression, segmentation, and classification gradually decreased and eventually stabilized, demonstrating that the model successfully learned the target object features without exhibiting significant divergence. In addition, the precision and recall values consistently remained above 0.85 after the initial training stage, indicating reliable detection performance on unseen data. The

mAP50 metric reached approximately 0.90, while the mAP50-95 metric stabilized at around 0.80, reflecting strong localization and segmentation capability under varying object poses and environmental conditions. The relatively small gap between the training and validation curves suggests that overfitting was effectively controlled through data augmentation and early stopping strategies. Overall, these results confirm that the trained model is sufficiently robust for deployment in the proposed vision-guided robotic sorting system.

Following the training process, the optimized YOLOv8-Segmentation model was deployed on the Vision PC and integrated into the complete machine vision pipeline. During system operation, images acquired from the industrial camera first undergo the image-processing procedures described in Section 2.2.1. The preprocessing stage includes grayscale conversion, bilateral filtering, median filtering, background compensation, top-hat transformation, contrast enhancement, and image binarization. These operations are applied to suppress image noise, reduce the effects of uneven illumination and metallic reflections, and enhance the visibility of object boundaries. As a result, the image quality is improved before being processed by the deep learning model.

After preprocessing, the image is forwarded to the YOLOv8-Segmentation network presented in Section 2.2.2. The trained model performs object recognition and instance segmentation simultaneously, allowing each object within the camera field of view to be identified and separated individually. Unlike conventional object detection methods that only provide bounding boxes, the segmentation model generates pixel-level masks for each detected instance. These masks provide detailed geometric information about the shape and contour of the target object, which is essential for subsequent localization and pose estimation tasks.

Once the segmentation masks are obtained, a series of post-processing operations are performed to eliminate invalid detections and extract the geometric features required for robotic manipulation. The segmented mask of each valid M8×15 mm bolt is analyzed to determine its spatial characteristics. The centroid of the mask is calculated to represent the object's position in the image coordinate system, while the Principal Component Analysis (PCA) algorithm described in Section 2.2.3 is applied to determine the object's principal axis. By analyzing the covariance matrix of the mask pixels, PCA identifies the dominant direction of the bolt body and calculates the corresponding orientation angle (θ). This angle represents the rotational pose of the object relative to the camera coordinate system and serves as an important parameter for robotic grasp planning.

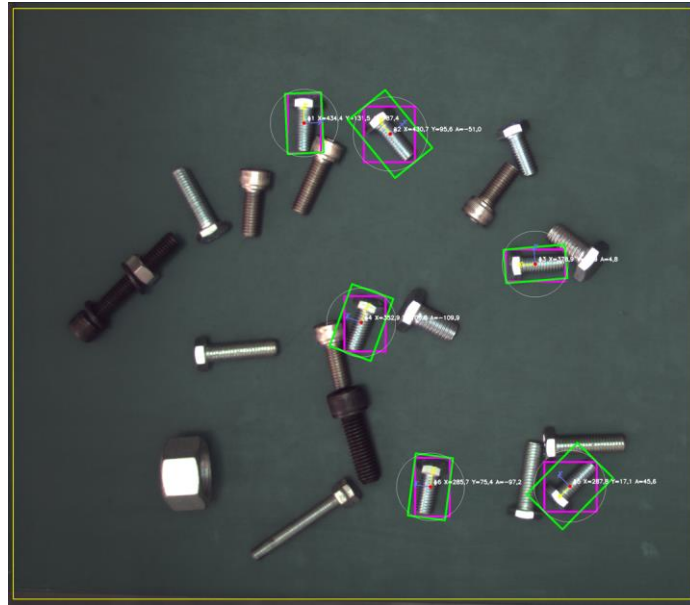


Figure 3.5 PCA processing result

Figure 3.5 presents an example of the PCA processing result, where the centroid and principal axis are extracted from the segmentation mask. The centroid provides the position of the object, while the principal axis indicates its orientation. Together, these parameters define the pose of the target bolt within the image frame. The extracted centroid coordinates are subsequently transformed into real-world robot coordinates using the Homography calibration model, while the orientation angle is transmitted directly to the robot controller. Through the integration of image preprocessing, YOLOv8-based instance segmentation, geometric post-processing, and PCA-based orientation estimation, the proposed vision system is capable of providing accurate position and orientation information for reliable robotic pick-and-place operations.

3.3 Coordinate transformation design

To implement the coordinate calibration theory presented in Section 2.2.4, a Homography transformation matrix was established to map coordinates from the camera image plane to the robot workspace coordinate system. Since the industrial camera was rigidly mounted above the Flex Feeder and the working surface remained planar, the Homography method could be applied to determine the relationship between image coordinates and real-world coordinates.

The calibration process was performed by collecting corresponding point pairs from both coordinate systems. First, the camera was fixed above the feeding tray to ensure complete coverage of the robot picking area. The Nachi robot was then switched to Teach mode, allowing the robot end-effector to be manually positioned at predefined calibration points distributed across the workspace. At each calibration point, the physical coordinates (X, Y) were recorded directly from the robot controller, while the corresponding image coordinates (u, v) were extracted from the camera image. To

improve calibration accuracy and reduce transformation errors across the entire workspace, a total of 32 reference points were uniformly distributed over the robot picking plane.

Figure 3.6 illustrates the calibrated camera field of view and the coordinate acquisition process performed by the robot. After collecting all corresponding point pairs, the two datasets were used to calculate the Homography transformation matrix according to the mathematical model described in Section 2.2.4. The resulting matrix enables the conversion of object centroids obtained from the vision system into robot workspace coordinates expressed in millimeters.



Figure 3.6 Calibrated camera field of view and robotic arm taking points on physical tray

The calibration accuracy was subsequently evaluated using an independent set of validation points that were not included in the matrix generation process. For each

validation point, the image coordinates were transformed using the computed Homography matrix and compared with the actual robot coordinates. Experimental results showed a maximum positioning error of approximately 0.3 mm within the robot coordinate system, demonstrating that the calibration method provides sufficient accuracy for the proposed vision-guided robotic picking application.

3.4 Flex feeder operational strategy design

3.4.1 Component spreading mode

The component spreading mode is triggered when the Vision PC fails to find valid objects to pick or when the component distribution density on the vibrating tray is unsuitable for proper recognition. The goal of this mode is to re-disperse parts across the feeder surface to establish a more favorable distribution state for the camera and robot in the next cycle.

The control program follows a sequential mechanism composed of two main blocks: Up Feeder and Down Feeder. In the initial stage, the PLC activates the Up Feeder to run a vibration sequence to lift and distribute components onto the feeding tray. The vibration runtime is maintained for a specific duration via Timers and internal status flags to ensure components are spread evenly across the surface.

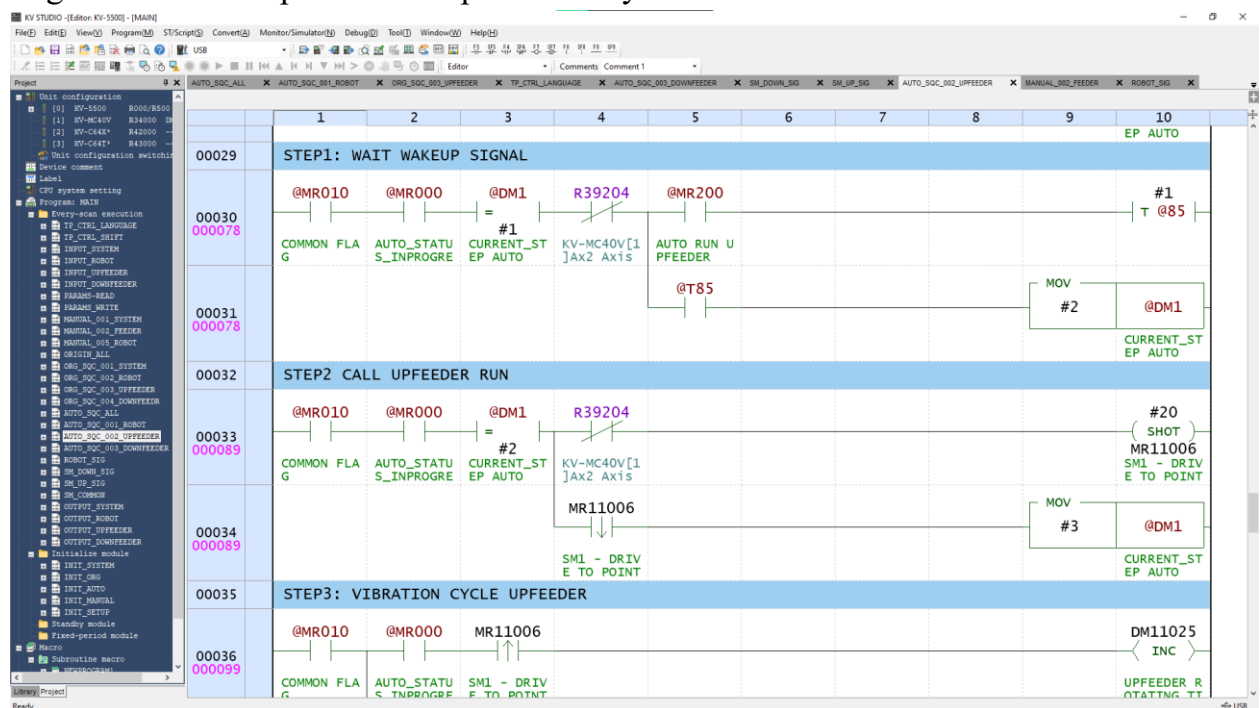


Figure 3.7 Ladder logic control program for Up Feeder component distribution

Once the feeding vibration finishes, the PLC controls the Down Feeder to move to pre-assigned points to assist in spreading and redistributing components. Position control is executed step-by-step through state sequences, combining Home sensor signals and internal control bits to verify limit-travel conditions before proceeding to subsequent steps.

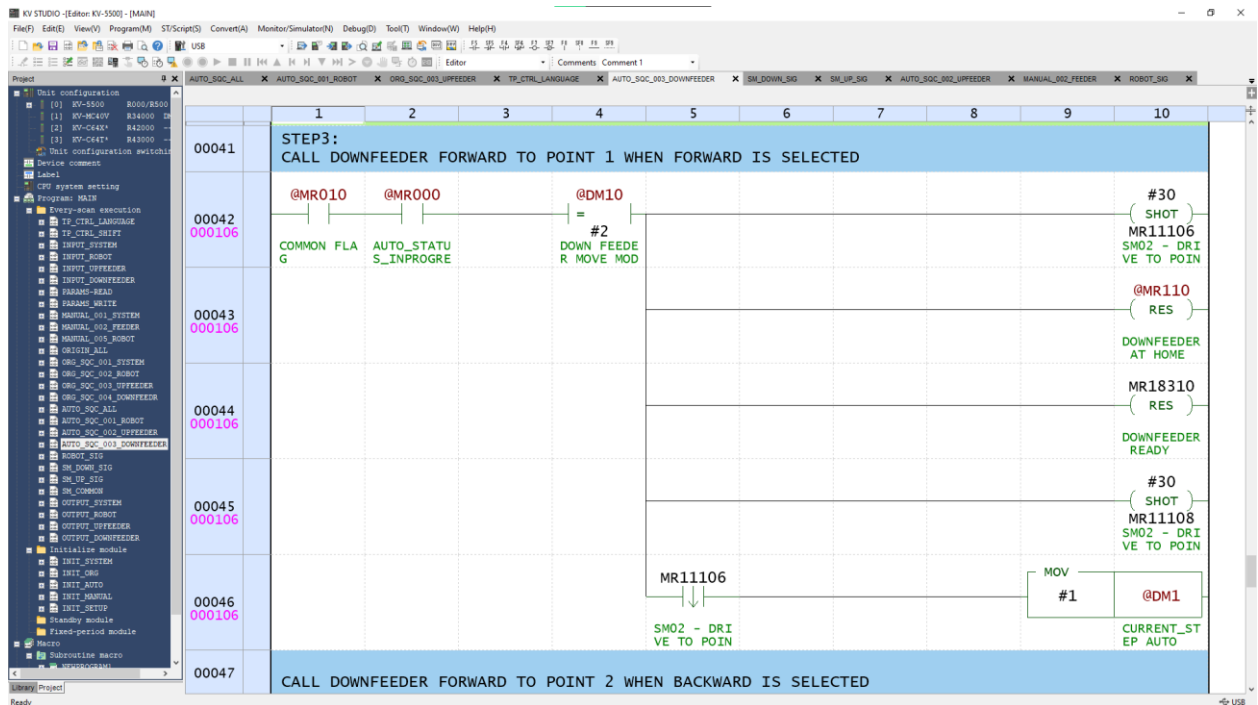


Figure 3.8 Ladder logic control program for Down Feeder component spreading

During operation, the PLC continuously updates the system's active state via registers and control flags to ensure the operational sequence between the Up Feeder and Down Feeder remains synchronized. Once the spreading cycle finishes, the system returns to a standby state, waiting for the robot to issue a new recognition request to the Vision PC. The spreading mechanism mitigates part overlap and occlusion on the vibrating tray, enhancing machine vision recognition and improving automatic picking cycle stability.

3.4.2 Purge mode

Purge mode is utilized to clear remaining workpieces from the vibrating tray after the robot finishes its picking list or when the system no longer detects valid target components. The purpose is to eject overlapping, jammed, or out-of-bounds components located away from the camera's optimal view, preparing the tray for the next feeding sequence.

In the PLC program, when the robot transmits a purge request signal, the PLC activates the Down Feeder mechanism to move toward the discharge position. This movement executes through sequential control states, combining the DownFeeder Home sensor signal and Timers to ensure the mechanism completes its travel profile before moving to the next sequence phase.

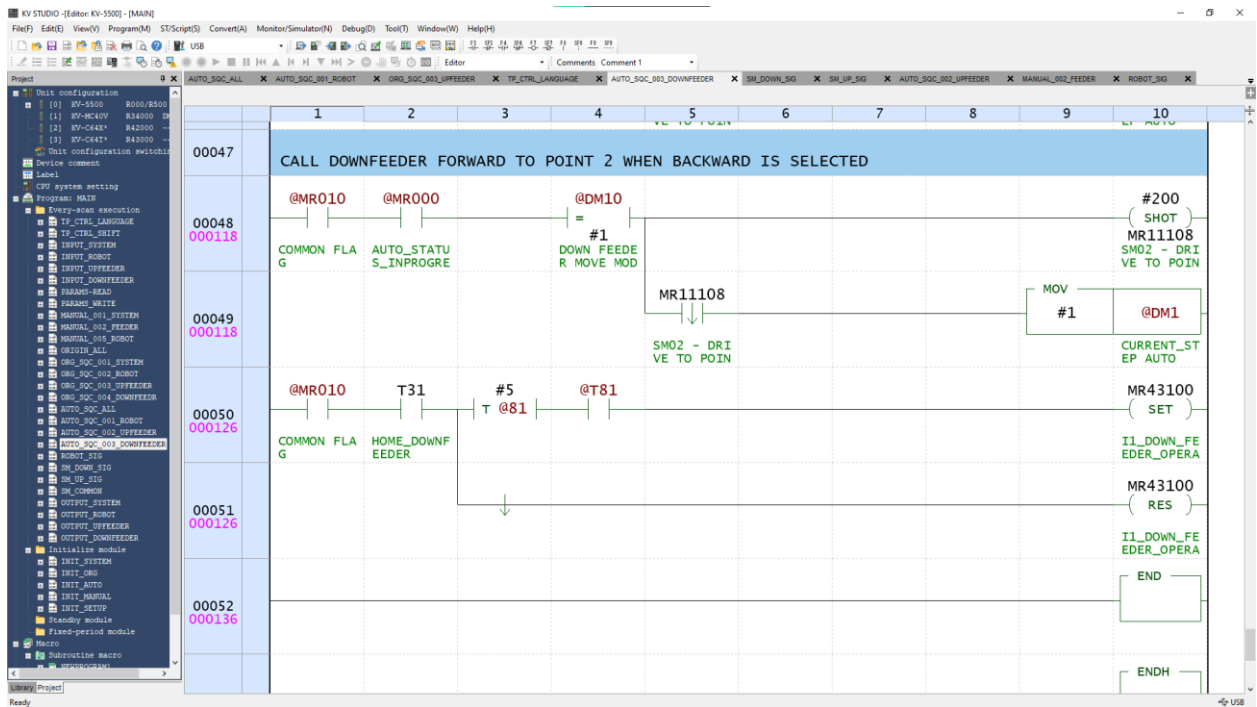


Figure 3.9 Ladder logic control program for cleaning Down Feeder components

After the purging routine finishes, the PLC resets status flags and returns the system to a standby state, ready for the next recognition and picking loop. This mechanism maintains feeding stability and prevents component accumulation on the tray during continuous operation.

3.5 Interface and data transmission design

To ensure monitoring and operational synchronization among system components, functional blocks are linked via a multi-tiered industrial communication architecture. The robot and Vision PC exchange data via socket communication, where the robot transmits a recognition request signal and receives back a coordinate string for picking. The Keyence KV-5500 PLC connects to the HMI VT5-W07 display via Ethernet TCP/IP protocol to facilitate status monitoring, parameter adjustments, and sequence tracking. Configuration on KV Studio (V11) and VT Studio (V8) follows these sequential steps:

Step 1: CPU Port Configuration in KV Studio: Activate the integrated Ethernet port on the KV-5500 CPU. Configure a static IP address and define logical communication ports. The system uses standard default Keyence ports (such as Port 8500 or 8502) to optimize handshake speeds.

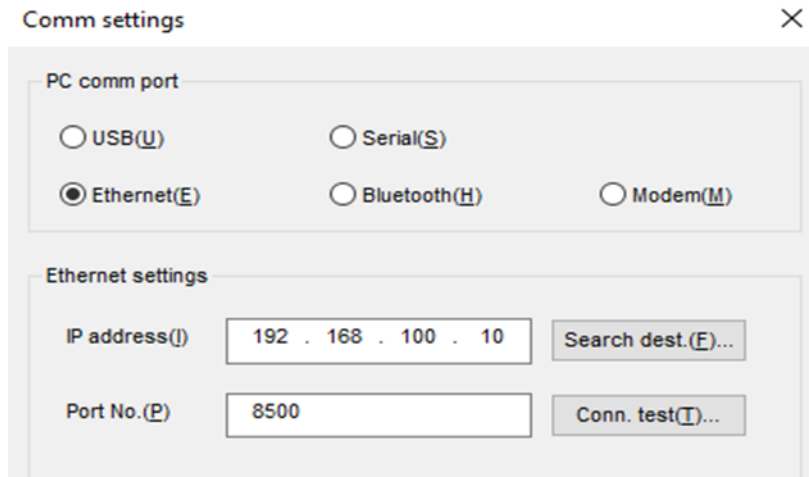


Figure 3.10 Communication settings panel for KV studio IP and port configuration

Step 2: Connection Conditions Configuration in VT Studio: On the VT Studio interface layout, initiate the peripheral communication parameters (PLC Communication Condition) via Ethernet TCP/IP. Enter the precise target destination parameters, including the IP address and Port code established for the PLC in Step 1, ensuring both devices identify each other on the local network.

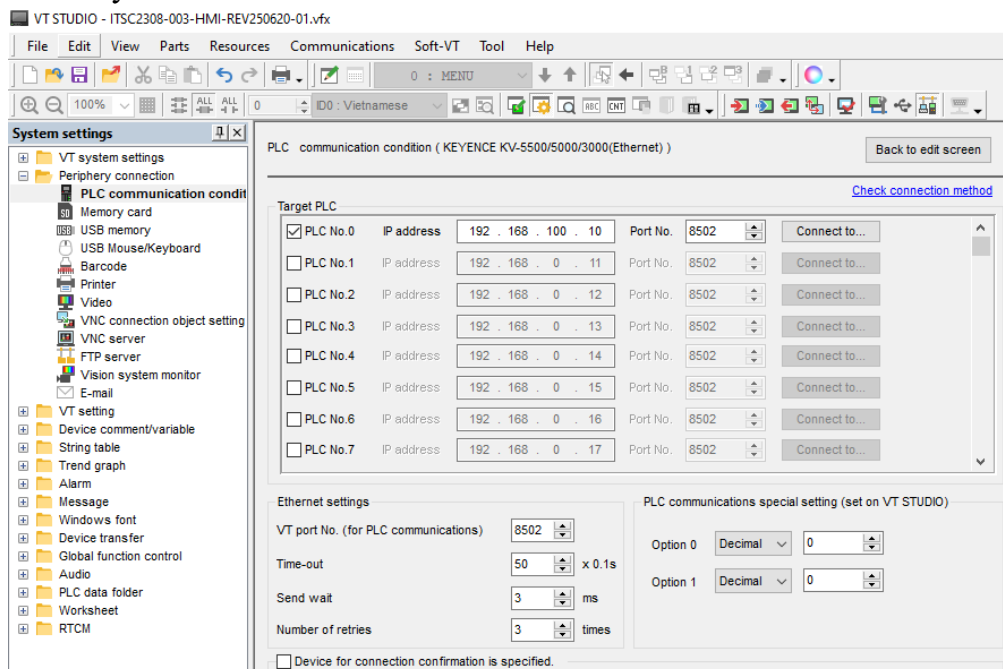


Figure 3.11 PLC communication condition setup page and Ethernet communication

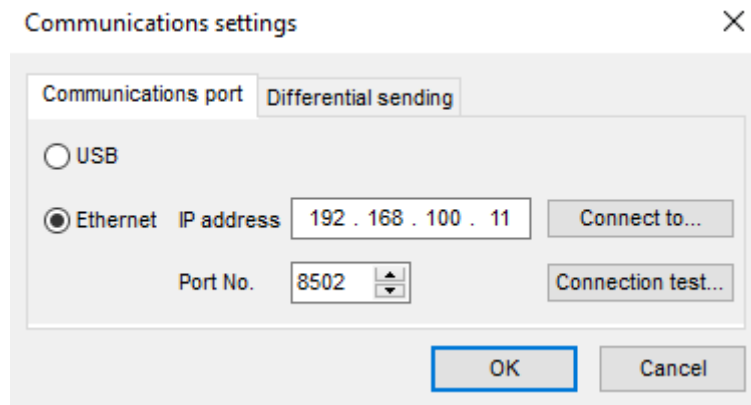


Figure 3.12 Ethernet communication setup in VT Studio

Step 3: Verification and System-Wide IP Synchronization: To guarantee stable data exchanges, the KV-5500 PLC and HMI VT5-W07 are assigned static IP addresses on the same Ethernet TCP/IP subnet mask. Similarly, the Vision PC and robot are configured on the same local network range to handle socket communication.

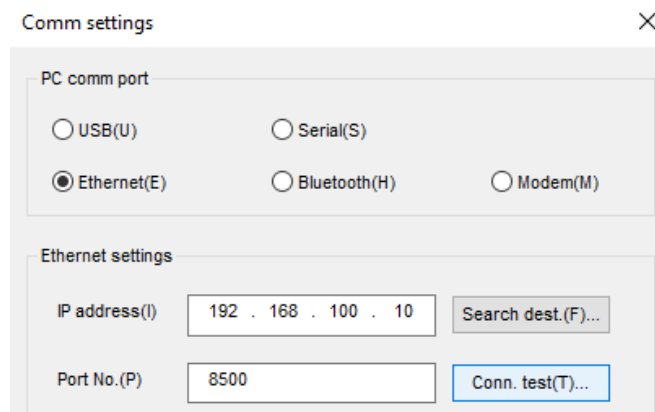


Figure 3.13 Communication settings for connecting with the PLC

Important Note: The static IP addresses of the PLC, HMI, and PC must reside on the same subnet network (differing only in the final octet). The configuration environment utilized includes:

- Windows 10 x64 host PC: IP 192.168.0.103
- Keyence KV-5500 PLC: IP 192.168.100.10
- HMI VT5-W07: IP 192.168.100.11
- PLC Software: KV Studio Ver11; HMI Software: VT Studio Ver8

Socket communication:

Once coordinate values for detected items are acquired, they must be transmitted to the robot. Additionally, the robot must communicate with the vision algorithm to flag when it is ready to pick. To execute this across separate runtime programs, a Socket Communication interface is implemented on the robot controller to handle direct transfers. For the image processing routine, Flask creates an API endpoint. A user

interface is integrated to allow operators to track results, and a WinForms application streams parameter pairs to the API to run the vision algorithms.

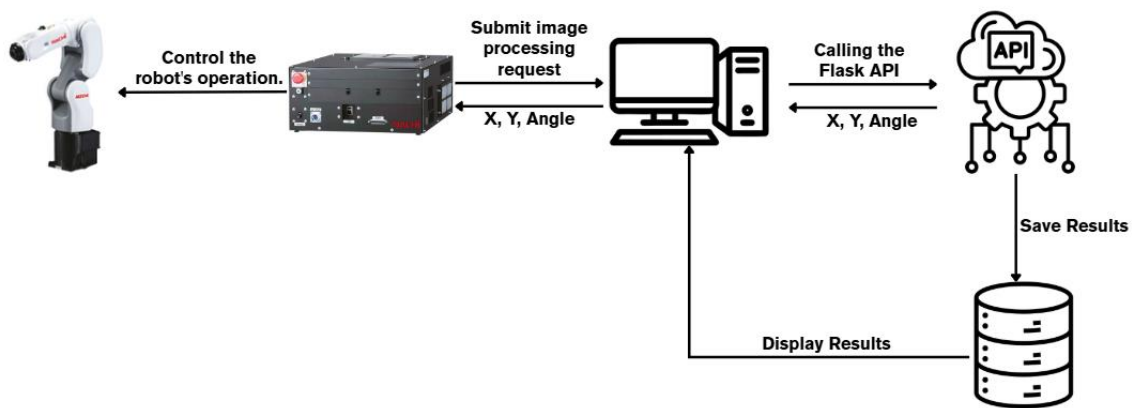


Figure 3.14 System data transmission diagram

The data transmission network layout includes:

- A Socket Server hosted on the host PC running the WinForms application.
- A Socket Client instantiated on the robot's CFD controller.
- An image processing API backed by the Flask Framework.

Data Exchange Workflow Steps:

- Step 1: The robot moves to its designated position, then calls its Socket Client program to send a request to the WinForms Socket Server, commanding an image capture and calling the image processing API to get part coordinates.
- Step 2: When the Socket Server receives the client request, it captures a frame, pulls parameters from the WinForms user interface as key-value pairs, and posts them to the image processing API.
- Step 3: The API receives the parameters and image to run processing algorithms. Upon completion, it saves the output image and returns coordinate and angular information back to the WinForms application.
- Step 4: Once the WinForms application receives the data successfully, the Socket Server transmits the coordinates and rotation angles of the workpieces to the Socket Client on the CFD controller, which subsequently drives the robot to the part location.
- Step 5: After completing part picking and placing components into a pallet array, the robot enters a standby state to wait for the next sequence request.

3.6 Supervision and control interface design (WinForm application)

This application centralizes all supervisory actions into a single window interface, enabling the operator to view real-time camera frames, track image processing outputs, and verify connection states for the Nachi MZ04 robot, Keyence KV-5500 PLC, and

intermediary services such as the Flask API or SQLite. Furthermore, it permits fine-tuning critical parameters including the ROI, binary thresholds, YOLO model selection, detection confidence, communication addresses, and robot safety options.

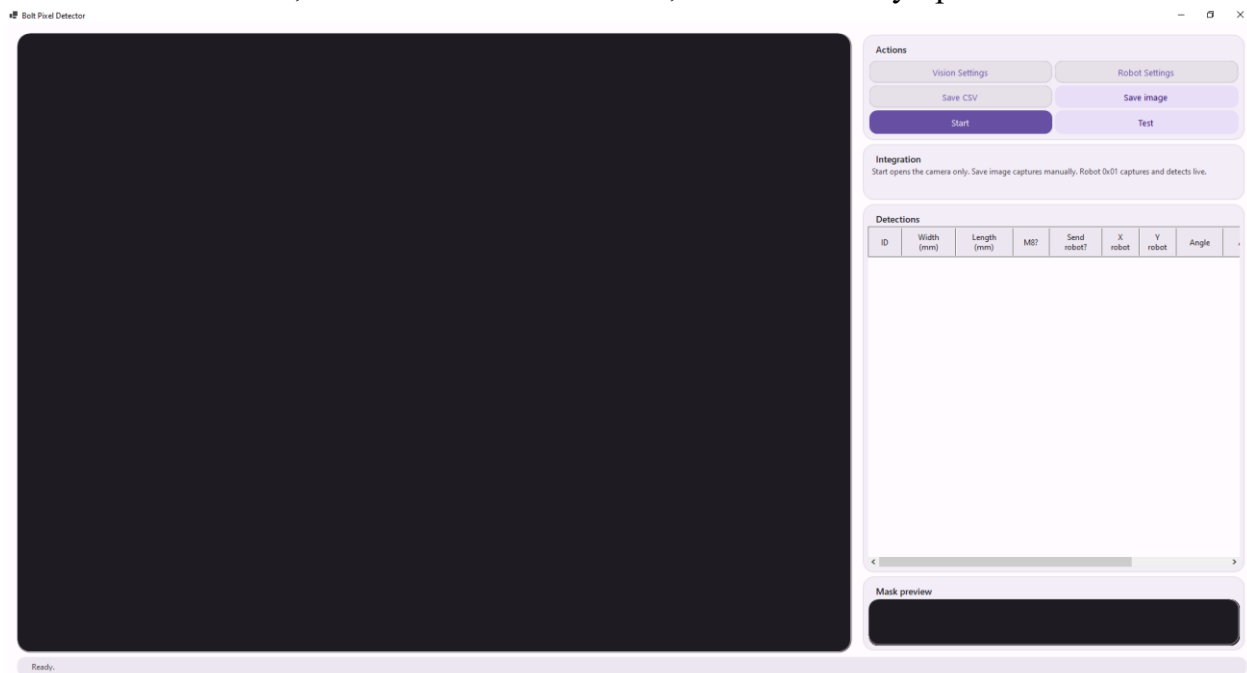


Figure 3.15 Main window layout of the WinForm application

3.6.1 Software architecture

The software architecture of the WinForm Application is organized into functional modules. The Camera Manager initializes and terminates connections with the Neptune camera, managing exposure times and frame captures; the Image Processing Module and OpenCV Processing Module execute pre-processing, binarization, geometric filtering, and contour extraction; the YOLOv8 Segmentation Module performs object segmentation inference; the Coordinate Transformation Module converts image coordinates into the robot's working coordinate system; the TCP Communication Module exchanges data with the Nachi controller via sockets; the Robot Data Management Module stores and synchronizes coordinates; the Logging & Alarm Module records system events, faults, and cycle times; while the User Interface Layer renders displays and accepts operator inputs.

The data flow begins at the industrial camera, passing through image processing blocks to compile a list of detected objects. Valid objects pass through size and robot safety filters, and are subsequently mapped to physical coordinates using Homography calibration. Final results are transmitted to the robot, logged into the database, and rendered on the interface for real-time tracking.

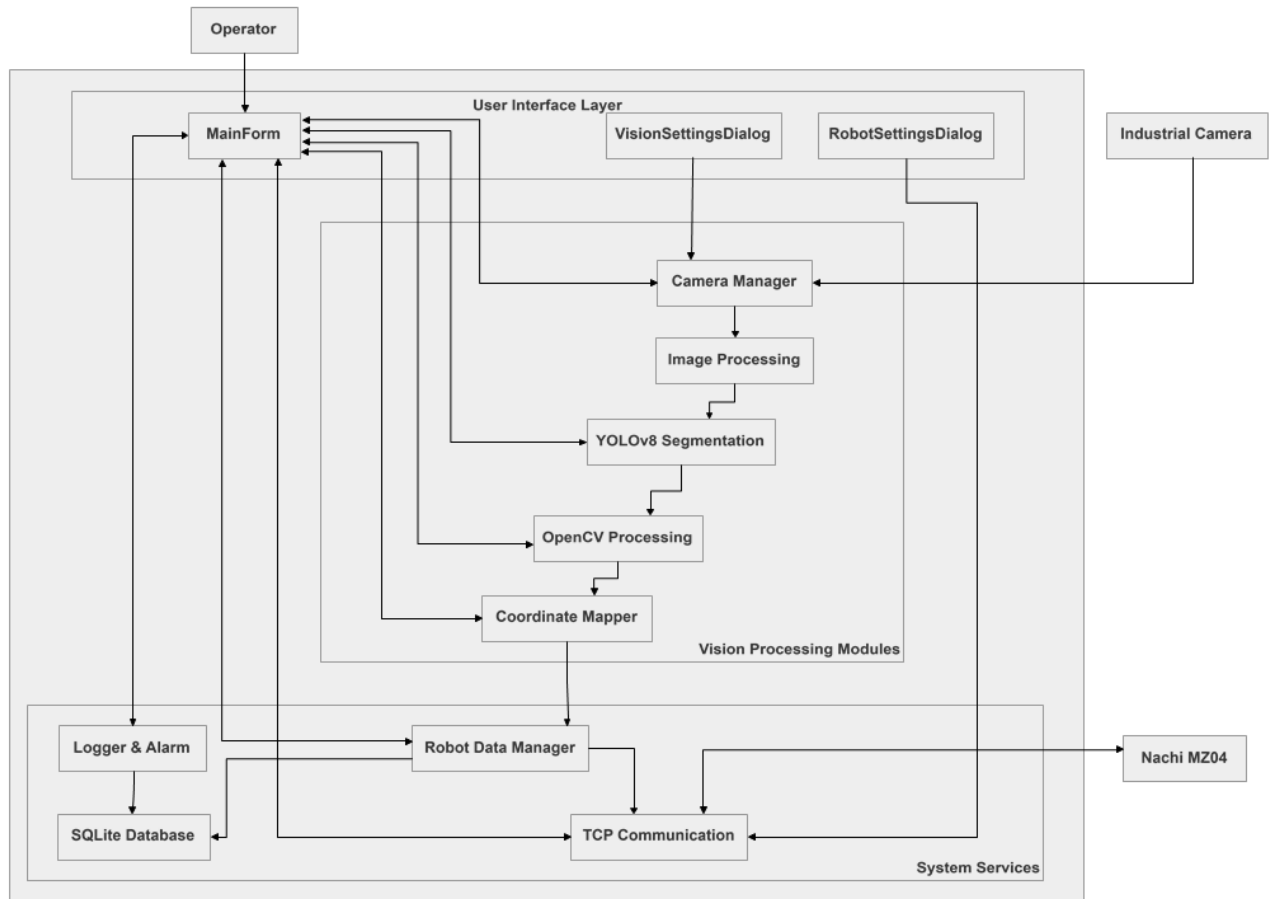


Figure 3.16 Module architecture block diagram of the WinForm application

3.6.2 Main interface layout

The main application interface features a two-region layout. The left region acts as the real-time camera stream display to monitor the feeding tray surface and image processing results. The right region acts as the control and monitoring hub, containing status readouts, a detection results table, a mask preview window, and operational button command clusters. This arrangement allows operators to monitor visual streams, processing results, and overall system status simultaneously on one screen.

The main stream preserves the geometric aspect ratio of the viewing area, while the mask preview box permits visual checks of binary or segmented regions post-processing. The detections table lists primary attributes for each object, including ID, width, length, M8 status, robot transmission permission, X-Y coordinates, rotation angle, area, confidence score, and circularity. Additionally, the bottom status bar displays consolidated readouts for processing times, system status, and diagnostic messages.



Figure 3.17 Geometric layout of the main user interface

3.6.3 Control and monitoring functions

Control functions are implemented as button commands on the main interface to simplify operations. The Start/Stop button initializes or terminates the camera link and begins or halts the monitoring cycle; Save image captures a frame, runs inference, and saves the annotated output; Test launches a camera validation dialog to view detection results without pushing data to the robot or database; Vision Settings and Robot Settings access respective configuration windows; while Save CSV exports the current detection grid results to an external file for calibration and analysis.

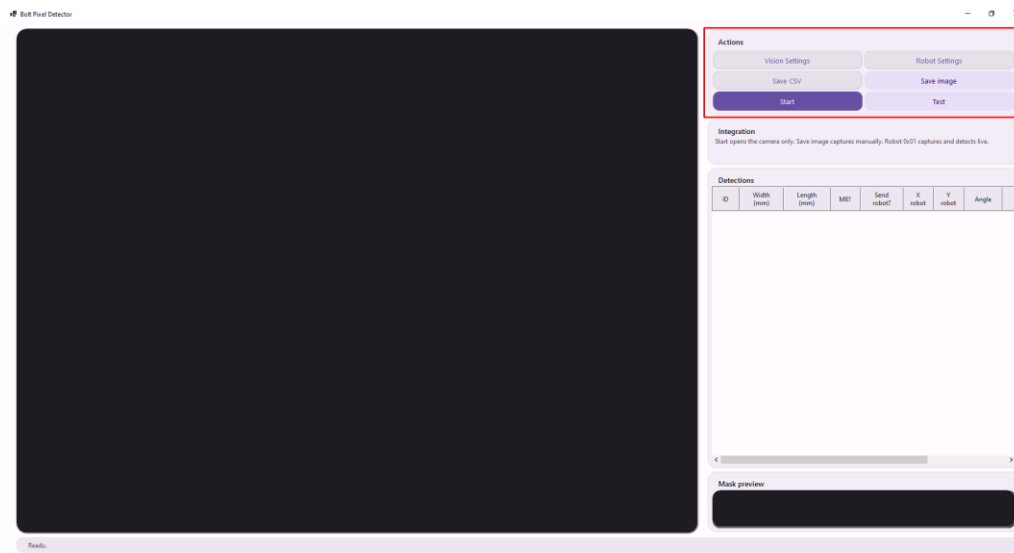


Figure 3.18 Control button region and configuration dialog access buttons

3.6.4 Robot coordinate data management

The image processing block output records geometric attributes for each detected object in the image, including its center point, area, rotation angle, bounding box, and

segmentation mask. VisionResult denotes data mapped to the robot coordinate system, and RobotCoordinateBatch is utilized when the robot requests a batch list of coordinates from the Vision PC within a single recognition cycle.

The detected coordinate list is simultaneously rendered in the results table and pushed into the communication queue. For each valid object, the system runs Homography coordinate transformation, adjusts the rotation angle to match robot conventions, and logs the entry into the Flask API and SQLite database. The sent to robot status flag marks whether a record has been retrieved by the robot, maintaining a FIFO mechanism and preventing duplicate data transfers across successive cycles.

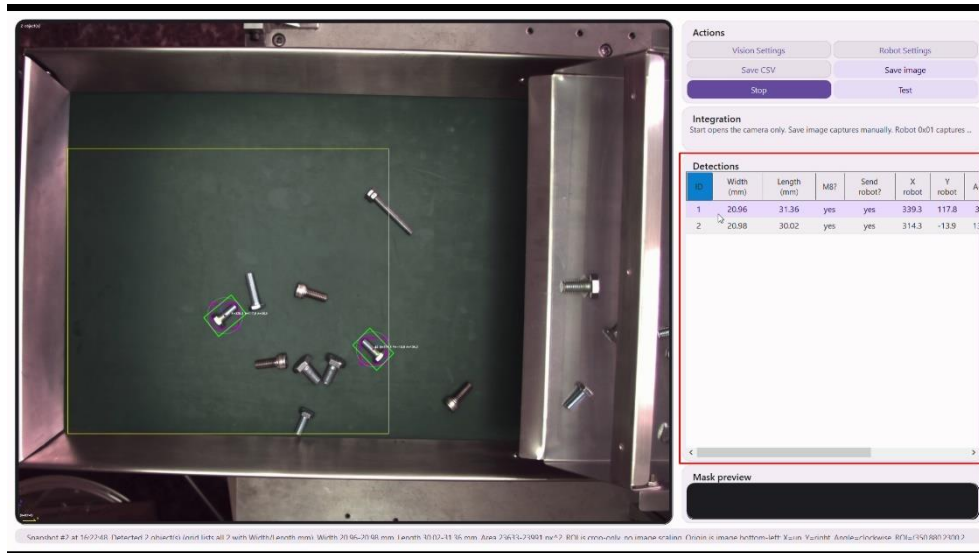


Figure 3.19 Detection coordinate list table and robot transmission status flags

3.6.5 Logging and alarm functions

The system is designed to log data in real time to assist in maintenance and fault diagnosis. The WinForm status bar shows consolidated messages regarding processing time, camera status, robot connection, pending object count, and database write outputs. Additionally, the system captures logs concerning TCP Sockets, recognition results, camera faults, robot errors, and database exceptions for incident tracing.

Logging history and cycle time records are essential for evaluating overall system performance. These records help determine image acquisition time, processing latency, communication lag, and robot response parameters, while easing image processing adjustments and fault diagnostics.



Figure 3.20 Status bar location and system alarm region

3.7 Robot arm control programming

The FD on Desk software is used to simulate robot motion trajectories and verify the feasibility of the control program before deployment on physical hardware. Simulation coverage includes verifying command sequences and checking relevant coordinates. Through simulation, physical system installation and calibration are carried out safely and efficiently.

Steps for Designing and Loading Robot Programs in FD on Desk:

Step 1: Launch the CFDonDesk online.exe application and enter the CFD controller's IP address to establish a link between the software and the robot controller.

Step 2: Select the appropriate operating mode: FD on Desk features VIEW, MONITOR, and OFFLINE modes. OFFLINE mode is chosen for programming and motion simulation since it operates independently of the physical controller hardware.

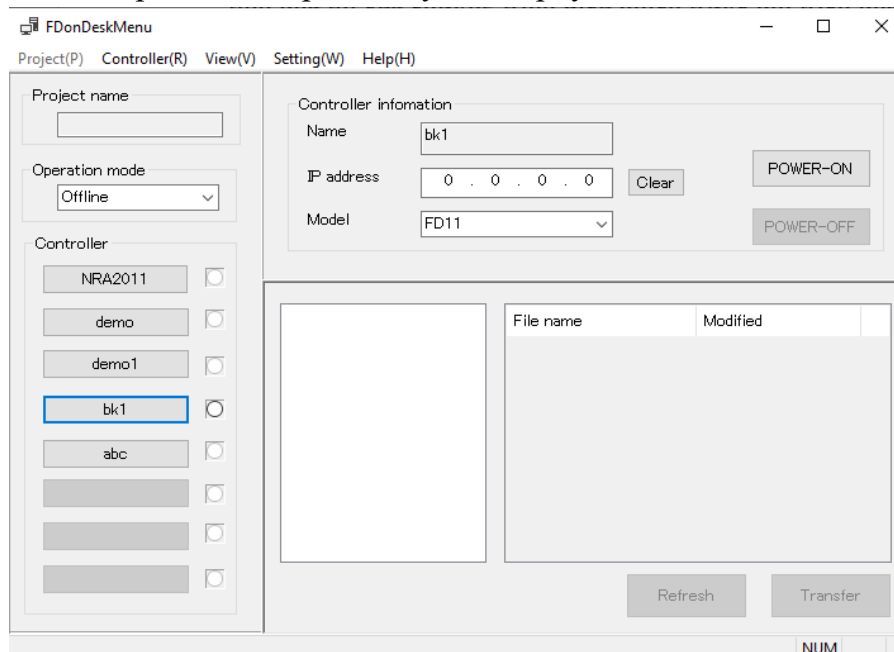


Figure 3.21 Start FD on Desk

Step 3: Prepare data and programs within FD on Desk: In the OFFLINE environment, users can edit robot programs, ASCII SLIM Robot Language scripts, and constant parameters. When simulating or teaching a program, Virtual TP, Virtual IO, and Virtual Robot utilities are utilized to construct workcells, assign tool/gripper properties, set taught points, and verify robot motion.

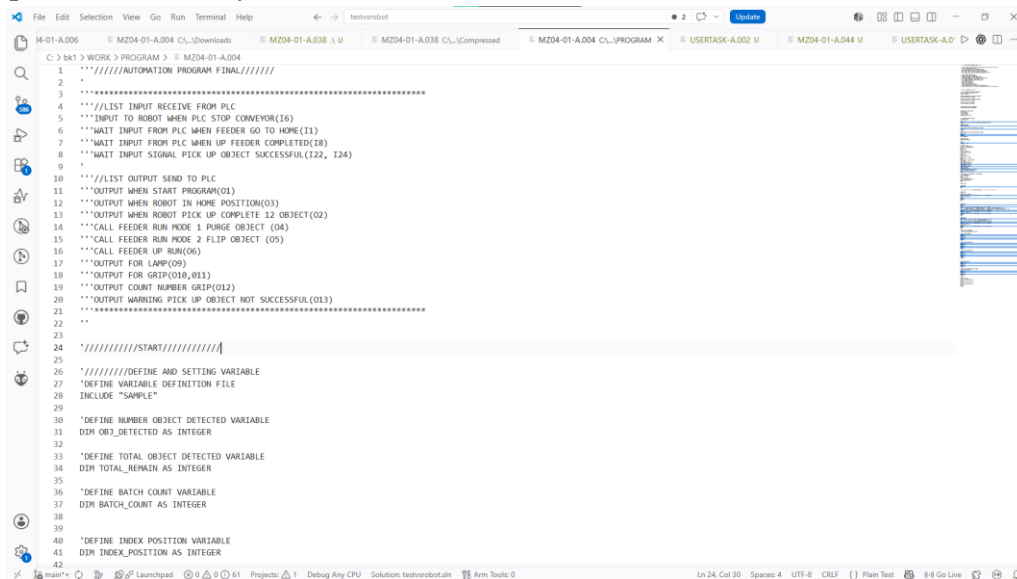


Figure 3.22 SLIM source code development interface in FD on Desk

Step 4: Transfer files from FD on Desk to the CFD controller: After completing code edits and running motion simulations in OFFLINE mode, the completed program is uploaded into the robot's CFD controller via FTP.

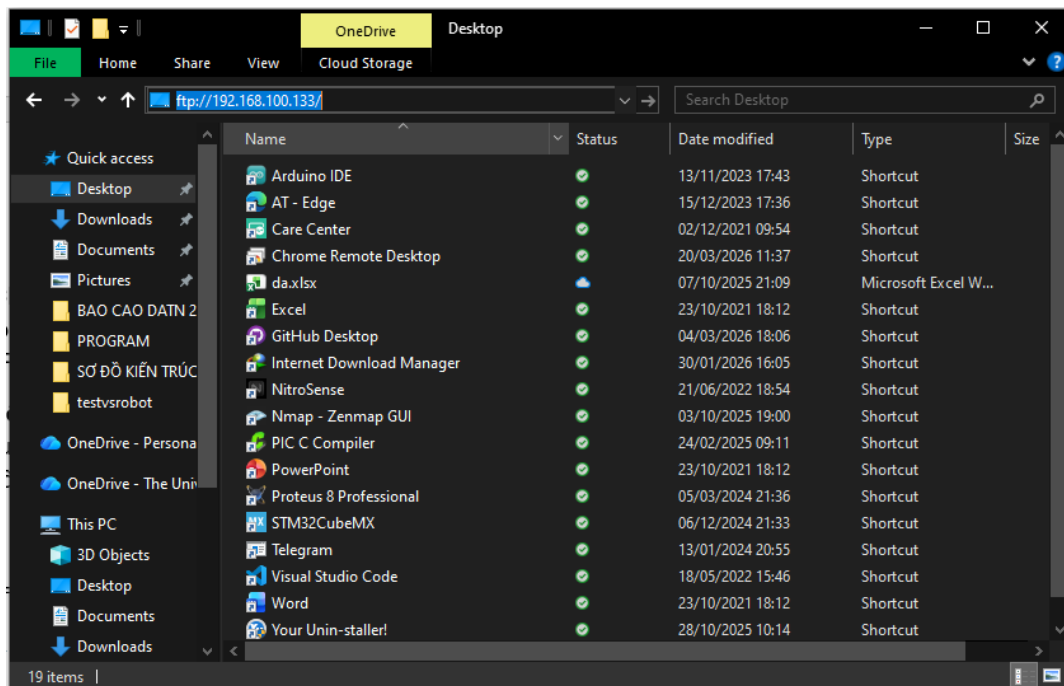


Figure 3.23 FTP transfer window displaying robot directories

Step 5: Final compilation and system verification:

Once data transfer completes, a physical Teach Pendant is utilized to compile the newly transferred program on the CFD controller.

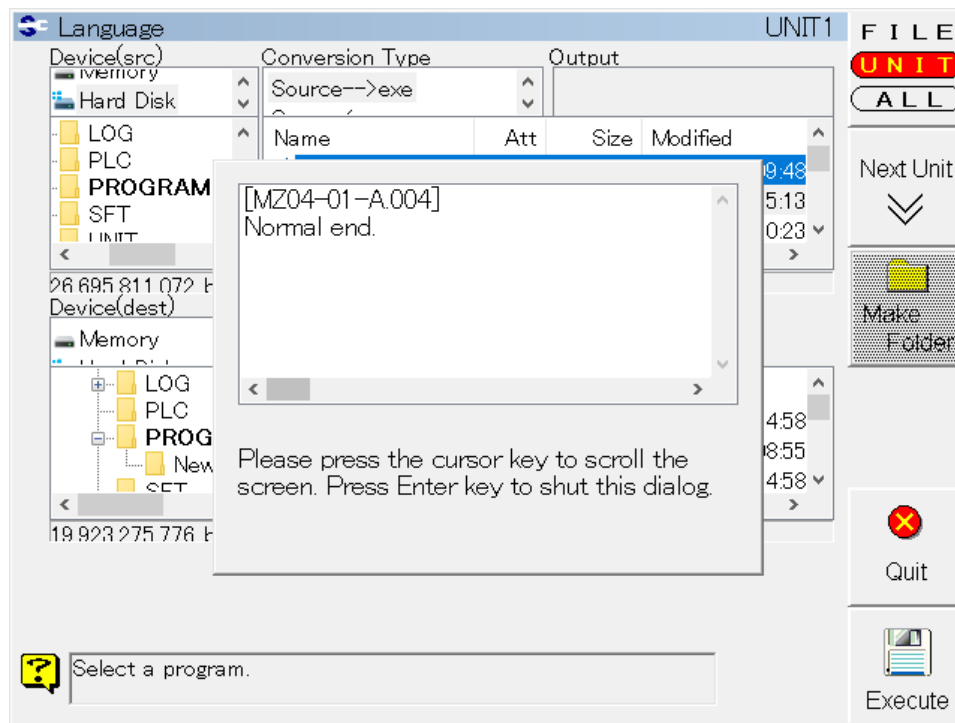


Figure 3.24 Teach Pendant compilation interface screen

3.8 Chapter summary

This chapter presented the design and implementation of the proposed vision-guided robotic sorting system. The overall system architecture, operational workflow, and communication framework were developed to ensure synchronized interaction among the Vision PC, PLC, Flex Feeder, and Nachi industrial robot. A YOLOv8-Segmentation-based machine vision system was designed and trained using a customized dataset to achieve reliable object recognition and instance segmentation in cluttered industrial environments. Furthermore, image preprocessing, geometric post-processing, and PCA-based orientation estimation were integrated to extract the centroid coordinates and rotation angle of target bolts. A Homography calibration procedure was implemented to transform image coordinates into robot workspace coordinates with sufficient positioning accuracy for robotic manipulation. In addition, Flex Feeder operating strategies, industrial communication mechanisms, supervisory software, and robot control programs were developed to support continuous and stable operation. The implementation results presented in this chapter establish the foundation for experimental validation and performance evaluation, which are discussed in the following chapter.

CHAPTER 4. SYSTEM DEVELOPMENT AND EXPERIMENTAL EVALUATION

4.1 Experimental setup and system integration



Figure 4.1 System hardware

Following the completion of the design and programming phases, the system was deployed as a physical experimental model to test synchronous operation among all functional blocks. Equipment interconnection and configuration strictly adhered to the architecture proposed in Chapter 3.

During integration, operational parameters including IP addresses, communication handshakes, Home positions, robot workspace limits, part-feeding modes, and image processing thresholds were calibrated under physical conditions. Following integration, trial runs were initiated to evaluate collaboration among the machine vision, robot control, and part-feeding blocks within the automated sorting loop.

4.2 Experimental results and technical parameter

Evaluation To evaluate system efficiency, experiments were performed on physical sample batches with the target object set as M8×15 mm bolts within a cluttered feeding distribution. Measured parameters are outlined below:

4.2.1 Positioning accuracy and sorting success rate evaluation

The system's spatial positioning accuracy was evaluated by analyzing deviations between actual physical coordinates and coordinates calculated by the Vision PC following Homography calibration. Experimental tests across 33 sample points yielded: Average RMSE: 1.166 mm - Satisfying the design constraint of $RMSE \leq 5$ mm

Maximum recorded error: 3.176 mm - Occurring at peripheral boundary points within the workspace

ID	Error (mm)	RANSAC Eval	Calculated coords (X, Y)
1	1.254	(Good)	(477.04, 83.44)
2	1.832	(Good)	(453.14, 201.79)
3	0.608	(Good)	(449.15, 145.79)
4	1.318	(Good)	(408.87, 196.99)
5	1.092	(Good)	(396.17, 76.04)
6	0.513	(Good)	(383.16, 140.04)
7	0.291	(Good)	(314.91, 90.85)
8	0.779	(Good)	(300.17, 201.20)
9	0.980	(Good)	(300.02, 151.26)
10	0.816	(Good)	(463.22, 123.99)
11	1.105	(Good)	(448.30, 44.17)
12	0.595	(Good)	(439.60, -4.07)
13	0.208	(Good)	(399.36, 87.04)
14	0.750	(Good)	(376.95, -18.25)
15	1.947	(Good)	(370.00, 138.45)
16	0.629	(Good)	(352.96, 34.55)
17	0.700	(Good)	(278.80, 103.66)
18	0.929	(Good)	(491.47, 114.09)
19	0.825	(Good)	(437.13, 174.09)
20	0.842	(Good)	(401.72, 19.99)
21	1.271	(Good)	(387.06, 87.09)
22	2.480	(Good)	(337.30, 171.07)
23	0.854	(Good)	(312.08, 89.28)
24	0.714	(Good)	(272.62, 156.75)
25	0.930	(Good)	(493.03, 149.22)
26	3.176	(Not Good)	(452.39, 91.89)
27	0.463	(Good)	(422.26, 30.70)
28	2.240	(Good)	(411.25, 190.27)
29	2.640	(Good)	(365.93, 125.96)
30	1.991	(Good)	(354.44, 29.31)
31	0.775	(Good)	(292.06, 207.85)
32	1.049	(Good)	(282.62, 133.41)
33	1.896	(Good)	(280.09, 34.66)

=====

SUMMARY OF ACCURACY RESULTS

=====

Total points: 33
 Inlier points: 32 (97.0%)
 RMSE: 1.166 mm
 Largest error: 3.176 mm (At ID 26)

=====

Figure 4.2 RMSE calculation

	A	B	C	D	E	F
1	Angle_vision	Angle_Robot				
2	-81.727	-84.15				
3	99.182	100.28				
4	-85.419	-85.62				
5	127.022	128.13				
6	131.045	132.37				
7	118.136	120.95				
8	-65.085	-65.05				
9	-98.975	-99.9				
10	94.903	94.11				
11	142.012	143.72				
12	-88.074	-88.074				
13	46.821	46.821				
14	153.872	153.872				
15	142.393	142.393				
16	28.045	28.045				
17	-172.44	-172.44				
18	121.714	121.714				
19	106.101	106.101				
20	174.498	174.498				
21	100.542	100.542				
22	27.625	27.625				
23	93.028	93.028				
24	24.993	24.993				
25	-84.887	-88.55				
26	-6.982	-5.31				
27	-86.107	-88.07				
28	-82.857	-84.58				
29	-82.549	-84.66				
30	-87.094	-86.45				
31	-89.314	-90.07				
32	-81.375	-83.28				
33	-81.135	-84.02				
34	Average	0.92975				

Figure 4.3 Calculate angular error

Object orientation angles extracted via PCA were evaluated against actual physical reference angles to assess rotational accuracy: The average angular error was 0.93° , meeting the stated technical requirements

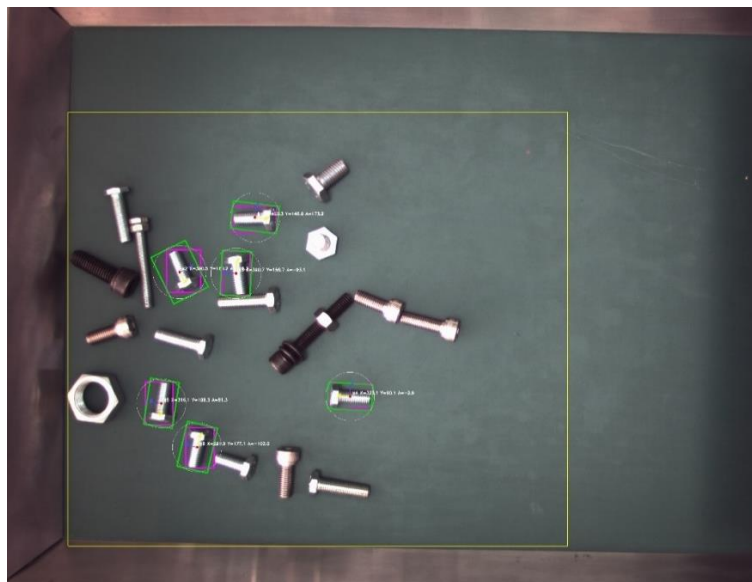


Figure 4.4 Image batch under vision analysis

Robot pick success rates were evaluated over 30 test batches, with each batch containing 20 target objects.

Across these trials, the system successfully executed 579 picks, while 21 objects were missed because the Vision PC failed to resolve the target within image processing routines. This equates to a 96.5% pick success rate, demonstrating stable coordination between the machine vision block and the robot during target localization and picking tasks.

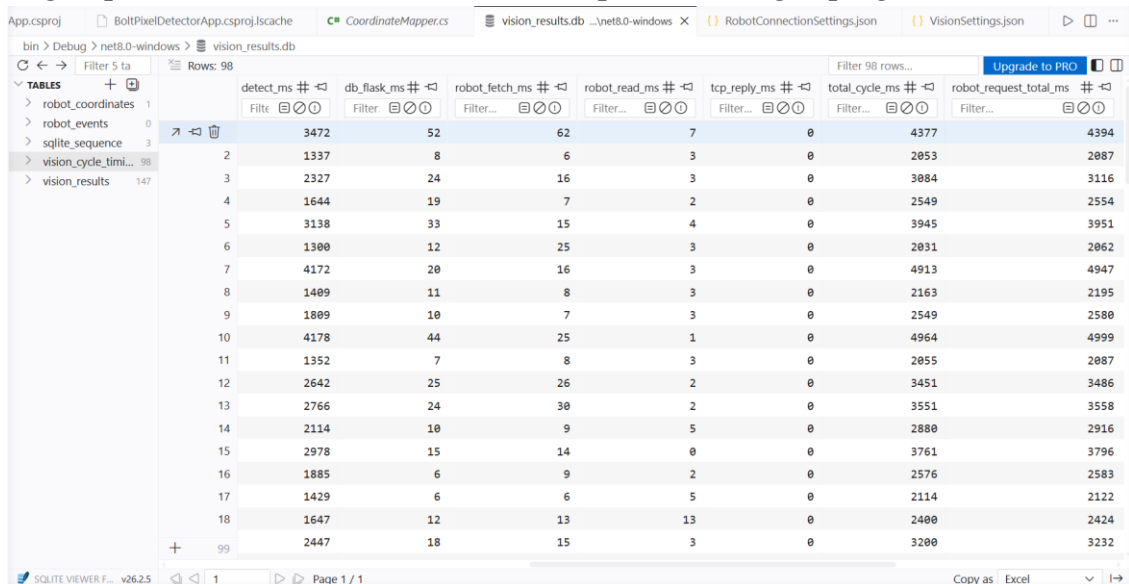
4.2.2 Flex feeder component separation efficiency evaluation

The separation efficiency of the Flex Feeder was evaluated across its two primary operational modes: component spreading and tray purging. In spreading mode, the PLC drives the Up Feeder and Down Feeder according to programmed cycles to mitigate part overlap and enhance camera visibility. The average time required for the system to transition the feeding tray from a cluttered stack to a uniformly spread state was 3 seconds.

In purge mode, the system was tested on its ability to clear remaining workpieces after the robot finished its picking sequence. The results showed an average purge duration of 8 seconds, reducing the residual part count on the feeding tray down to 0 items. Overall, the Flex Feeder satisfies the demand for stable component supply and supports the Vision PC's recognition routine across successive cycles.

4.2.3 Stability evaluation

System stability was evaluated based on communication consistency between the Robot, Vision PC, and PLC, alongside real-time cycle continuity. Over 70 continuous operational cycles, the system ran stably with an average communication latency of approximately 20 ms, without experiencing connection dropouts or data synchronization faults. The system was operated at 5% of maximum robot speed during testing to permit close observation of motion profiles and grasping behaviors.



	detect_ms #	db_flask_ms #	robot_fetch_ms #	robot_read_ms #	tcp_reply_ms #	total_cycle_ms #	robot_request_total_ms #
1	3472	52	62	7	0	4377	4394
2	1337	8	6	3	0	2053	2087
3	2327	24	16	3	0	3084	3116
4	1644	19	7	2	0	2549	2554
5	3138	33	15	4	0	3945	3951
6	1300	12	25	3	0	2031	2062
7	4172	20	16	3	0	4913	4947
8	1409	11	8	3	0	2163	2195
9	1809	10	7	3	0	2549	2580
10	4178	44	25	1	0	4964	4999
11	1352	7	8	3	0	2055	2087
12	2642	25	26	2	0	3451	3486
13	2766	24	30	2	0	3551	3558
14	2114	10	9	5	0	2880	2916
15	2978	15	14	0	0	3761	3796
16	1885	6	9	2	0	2576	2583
17	1429	6	6	5	0	2114	2122
18	1647	12	13	13	0	2400	2424
99	2447	18	15	3	0	3200	3232

Figure 4.5 Database query log showing cycle times and latency records

4.3 Project Conclusion and future directions

4.3.1. Attained results and system limitations

Overall, the proposed system has demonstrated the feasibility of integrating a Flex Feeder, a Vision PC, and an industrial robot for automated bolt sorting. Experimental results confirmed that the system can successfully perform object recognition, localization, orientation estimation, and automated pick-and-place operations for M8×15 mm bolts.

However, several limitations remain. The current system has only been developed and optimized for a single bolt type (M8×15 mm). In addition, recognition performance decreases under severe occlusion and remains sensitive to lighting variations. The overall cycle efficiency may also be affected when a large number of parts are present on the feeder tray or when communication delays occur between system components.

4.3.2. Future research and development scope in the future

In the future, the system can be expanded along the following directions to match flexible industrial manufacturing demands:

Investigate deep learning Artificial Intelligence (AI) algorithms for advanced data training to enable the system to automatically adapt to new part geometries.

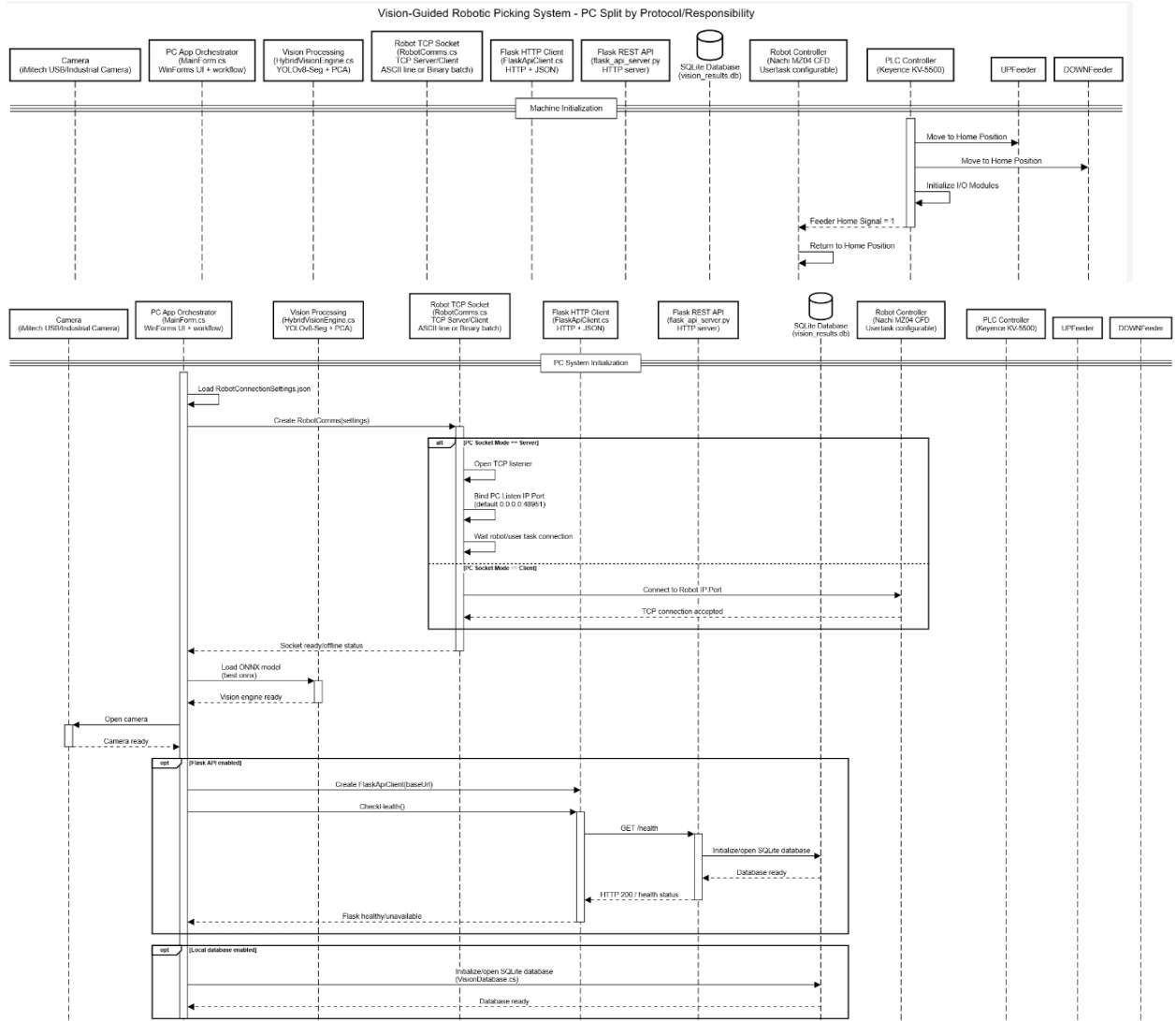
Develop advanced sorting algorithms to classify multiple distinct component types simultaneously within a single cycle, enhancing system flexibility and overall throughput.

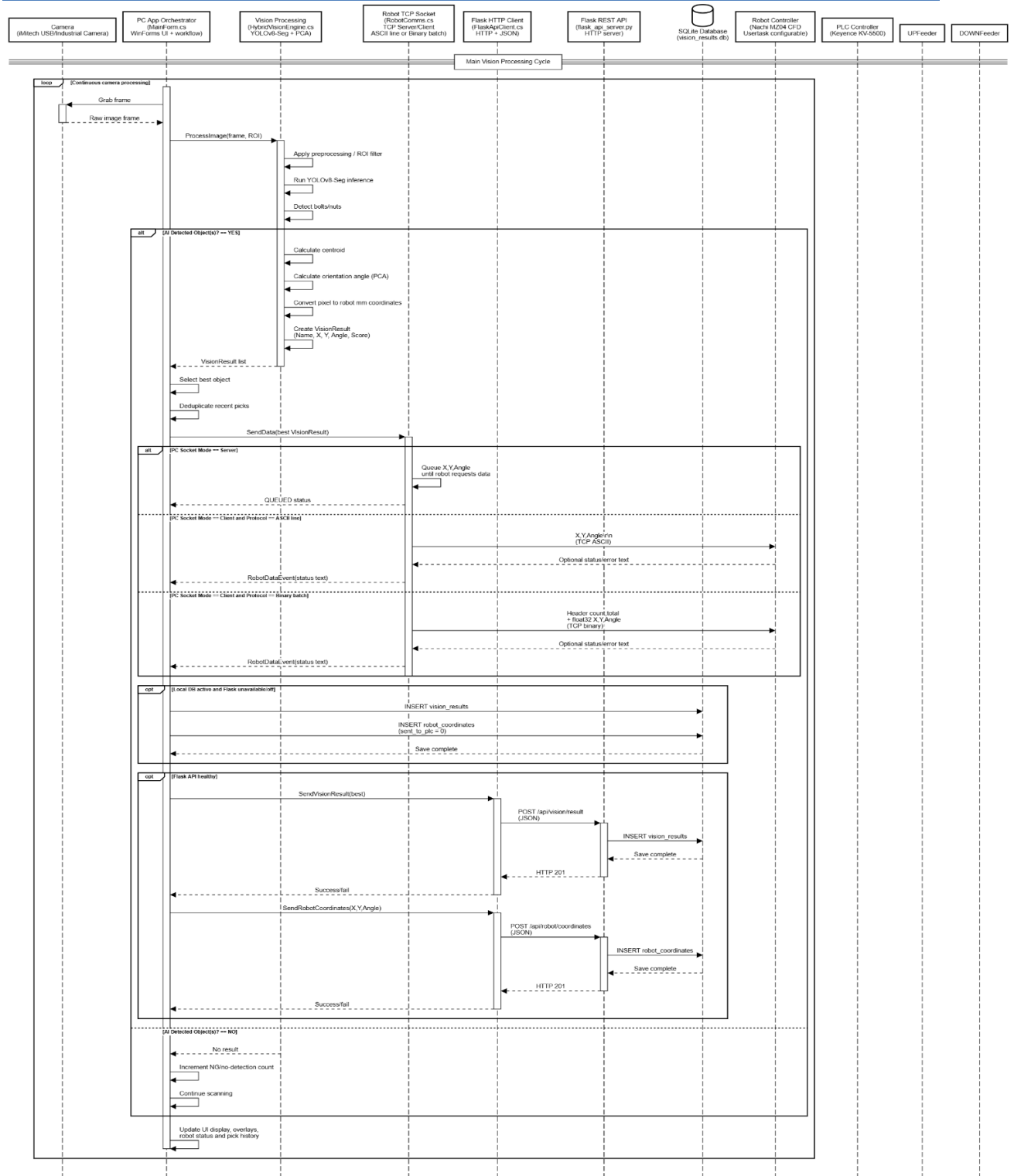
Design and construct a centralized SCADA platform or remote monitoring system to track the real-time status of the robot, PLC, Vision PC, and Flex Feeder, supporting fault alerts, data logging, and efficient operational management.

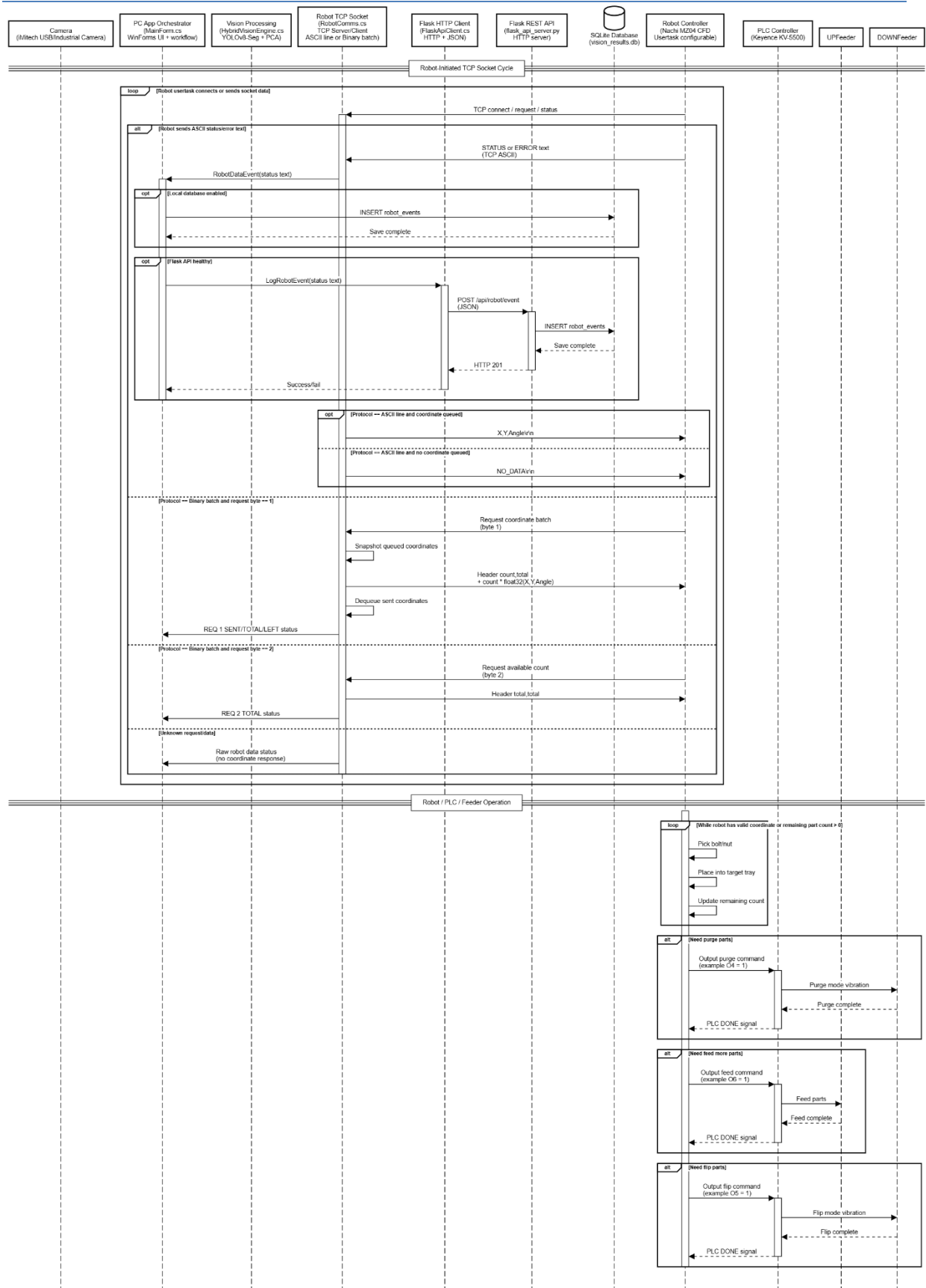
APPENDIX

State Diagrams and Execution Sequence:

The system executes a sequential cycle initiated and driven by the robot. When ready, the robot commands the camera to acquire an image. The vision computer analyzes the frame, identifies target components, and returns coordinates to the robot. The robot then decides whether to pick, request part feeding, or initiate a tray purge depending on real-time feedback.







REFERENCES

- [1] Lin, C. et al., “Automated Screw Inspection Using Vision-Based Methods,” *Applied Sciences*, vol. 11, no. 20, 2021.
- [2] M. (2020), “Flexible Feeding Systems in Industry,” *Procedia Manufacturing*.
- [3] A. T. Setiawan, Suwandi, H. L. Wijayanto, and A. W. Arohman, "Automated Sorting Conveyor Using Pneumatic Actuators for Industrial Applications at Morowali Metal Industry Polytechnic," *Jurnal Teknologi dan Manajemen*, vol. 22, no. 2, pp. 37–48, 2024. Zhang, Y. et al., “Machine Vision for Industrial Inspection,” *Applied Mechanics and Materials*, vol. 220–223.
- [4] Kovač, M. et al., “Vision-Guided Robotic Systems in Industrial Applications,” *Technologies*, vol. 12, no. 9.
- [5] Nachi-Fujikoshi Corp, *MZ04/MZ04E Robot Instruction Manual (Setup and Maintenance)*, 6th ed., Toyama, Japan, 2022.
- [6] J. Terven, D. M. Córdova-Esparza, and J. A. Romero-González, "A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS," *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, Nov. 2023..
- [7] A. Gambale, E. Kerr, D. Kerr, and S. Coleman, "Computing the Orientation of Hardware Components from Images Using Traditional Computer Vision Methods," *Engineering Proceedings*, vol. 65, no. 1, p. 8, Mar. 2024.
- [8] A. Giannoccaro, R. De Fazio, and P. Visconti, "An Innovative Vision-Guided Feeding System for Robotic Picking of Different-Shaped Industrial Components Randomly Arranged," *Technologies*, vol. 12, no. 9, p. 150, 2024.
- [9] Z. Zhang, L. Chen, and Q. Wu, "A Robust Eye-to-Hand Calibration Method for Vision-Guided Pick-and-Place Systems Using Least Squares," *Sensors*, vol. 23, no. 12, p. 5567, 2023.
- [10] Keyence Corporation, *KV-5500/5000/3000 Series Programmable Controller User's Manual*, Osaka, Japan, 2020.
- [11] X. Wang, Y. Zhang, and Z. Liu, "A Method to Improve the Accuracy of Hand-Eye Calibration Based on Data Filtering and Parameters Optimization," *IEEE Access*, vol. 12, pp. 24510-24520, 2024.
- [12] C. Tomasi and R. Manduchi, “Bilateral Filtering for Gray and Color Images,” in *Proceedings of the 1998 IEEE International Conference on Computer Vision*, Bombay, India, 1998, pp. 839–846.
- [13] OpenCV Documentation, “Smoothing Images” (tutorial covering Gaussian blur, median blur, and bilateral filtering).
- [14] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018.
- [15] OpenCV Documentation, “Morphological Transformations.”
- [16] N. Otsu, “A Threshold Selection Method from Gray-Level Histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.
- [17] OpenCV Documentation, “Image Thresholding.”